Lab 9. Continuous Testing

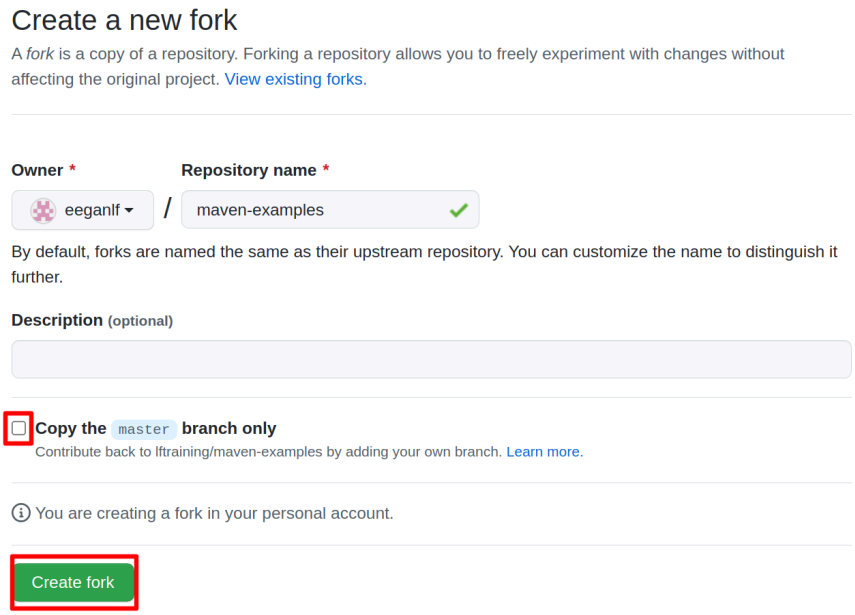
By the end of this lab exercise, you should be able to:

- Analyze code coverage with JaCoCo
- Improve the code coverage
- Set up SonarQube to automatically scan your repository
- Create a project gating system to automatically decide whether the code is safe to deploy
- Add integration and acceptance tests with Docker Compose
- Incorporate all of the steps into the Jenkins pipeline

Code Coverage with Jacoco

Create a new Jenkins project which will download the following sample repository and run unit tests with a Jacoco plugin for Java and display code coverage reports.

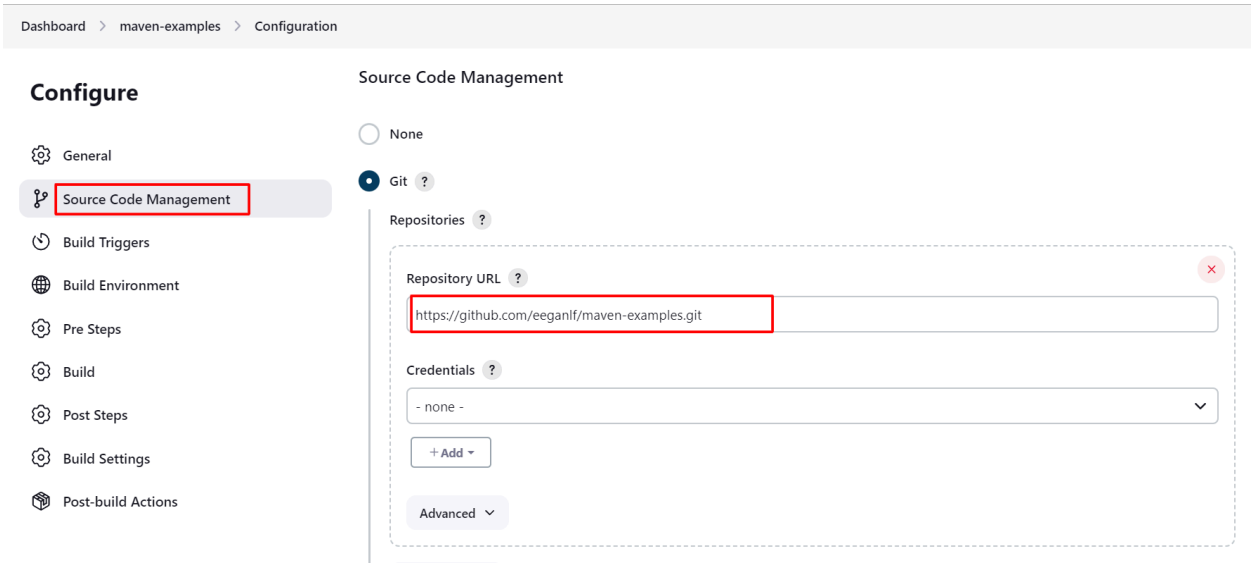
Fork the maven-examples repository to your GitHub account. Be sure to *uncheck* the **Copy the master branch only** option, as follows:



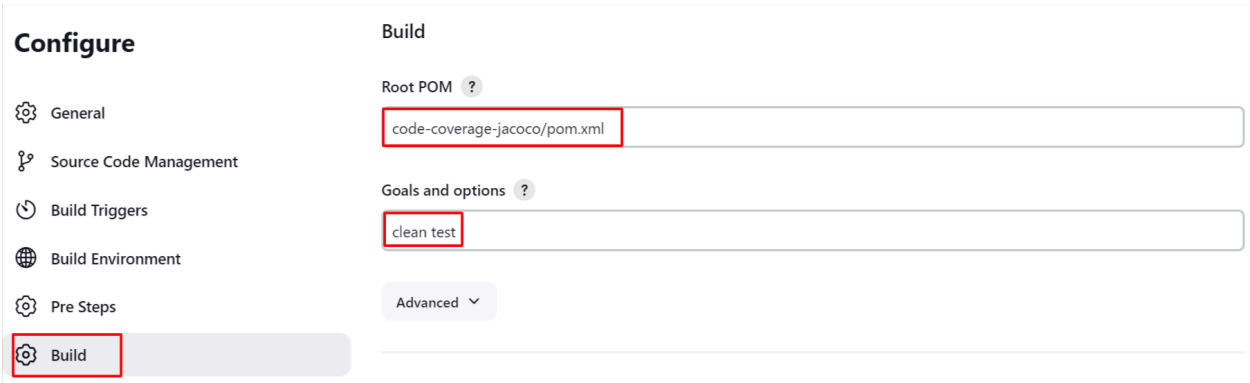
Install the **Coverage** Plugin for Jenkins.

Create a Maven project named **`maven-examples`**, provide the repository you forked as source, add Maven goals to run tests on it. Use “clean test” as the Maven goal.

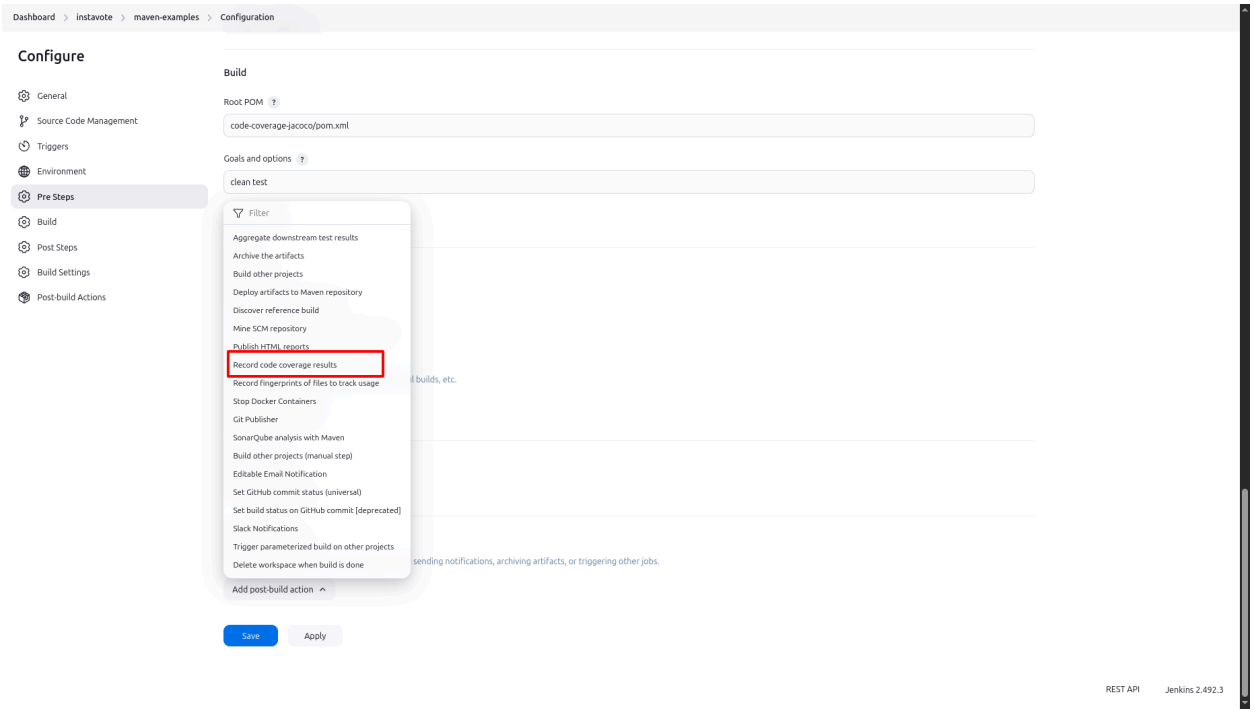
Under the **Source Code Management** section, select **Git** and paste the repository URL of the **`maven-examples`** repository that you forked.



Under the **Build** section, enter the path to the `pom.xml` as `code-coverage-jacoco/pom.xml`, and type **clean test** into the **Goals** text box.

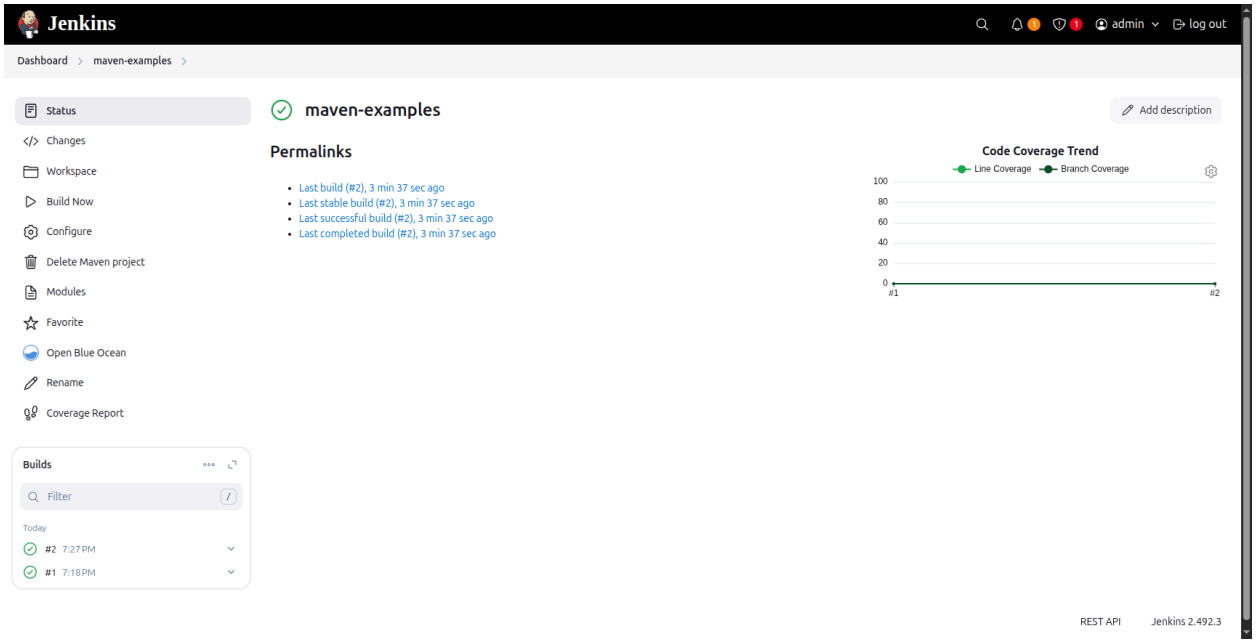


From post-build actions, choose **Record code coverage results**.



Select **JaCoCo Coverage Reports** as the **Coverage Parser** and **Save**.

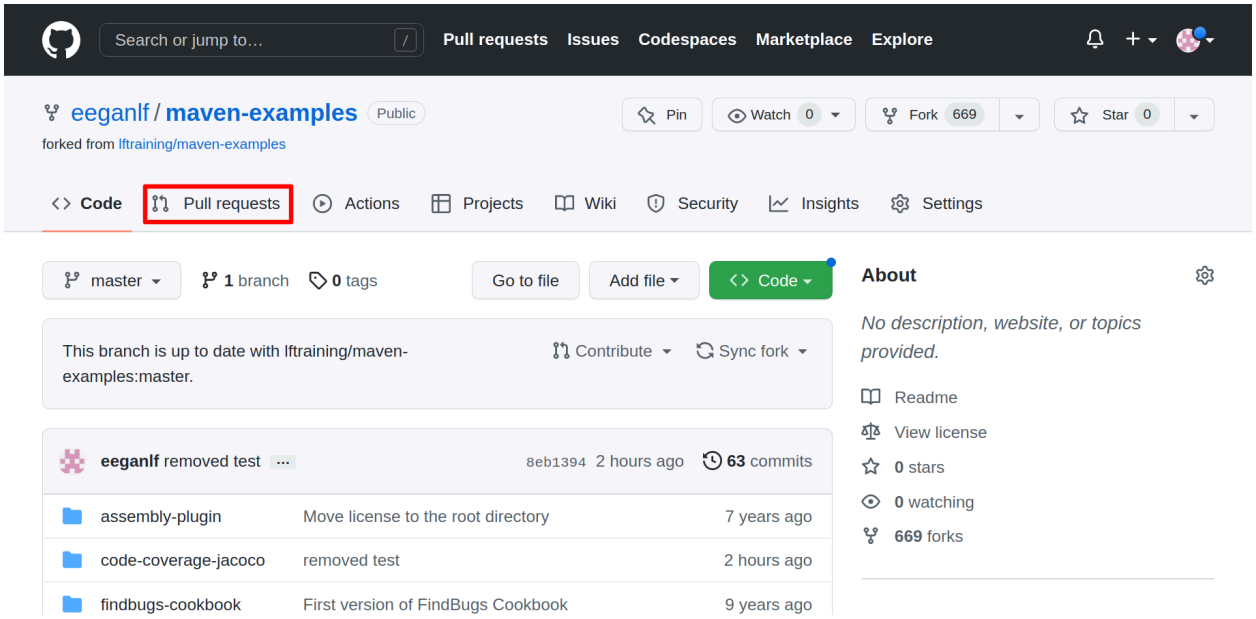
Build the project twice and refresh the page after the build is complete:



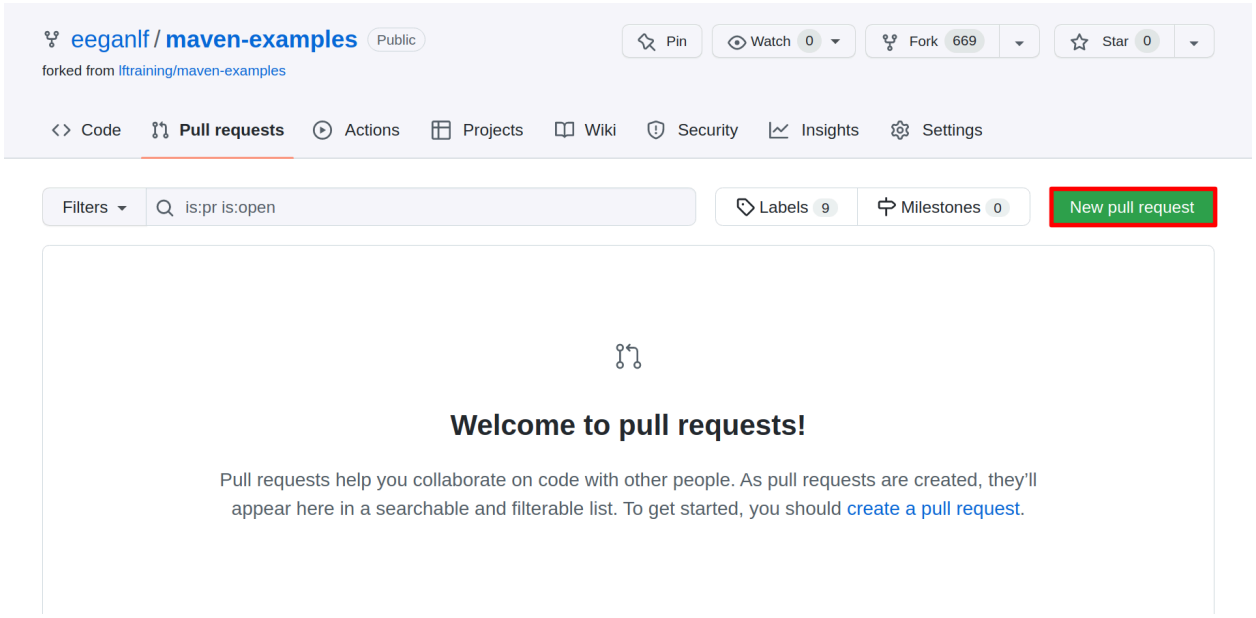
You should see a Code Coverage Trend between the two builds. It won't show much at the moment because you have only built the project once and there are no tests. You will add tests to the master branch next.

Create and merge a pull request from the `ut1` branch in your repository onto your `master` branch.

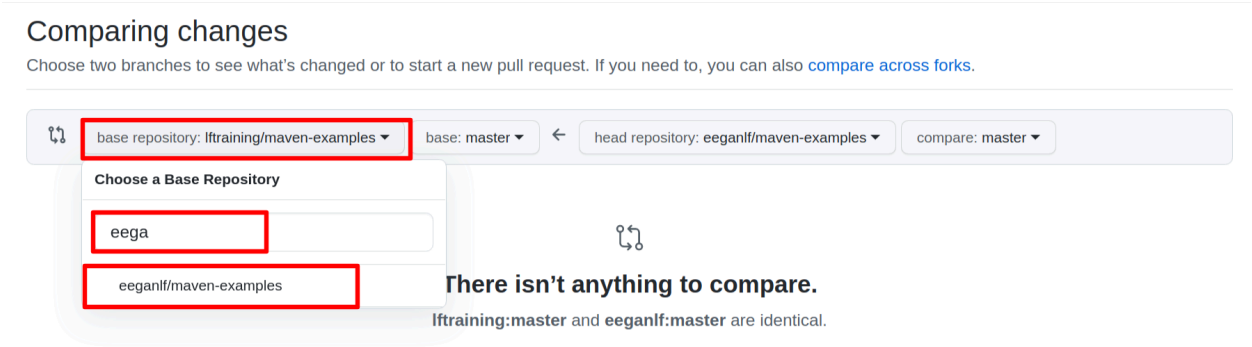
First, navigate to **Pull requests**:



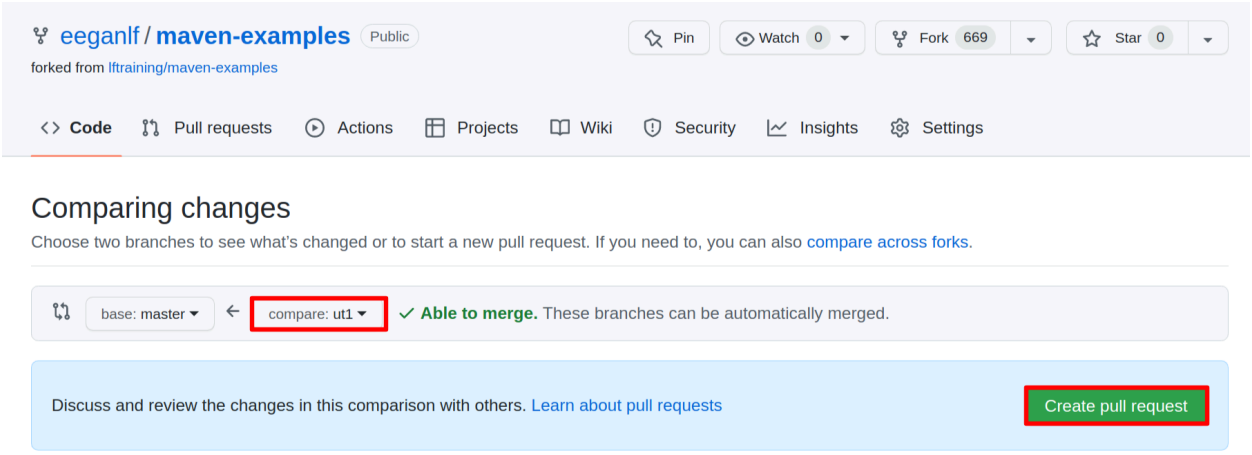
Click **New pull request**:



Click on the **base repository** dropdown and search for your GitHub username. When you find your repository, select it as the base repository:



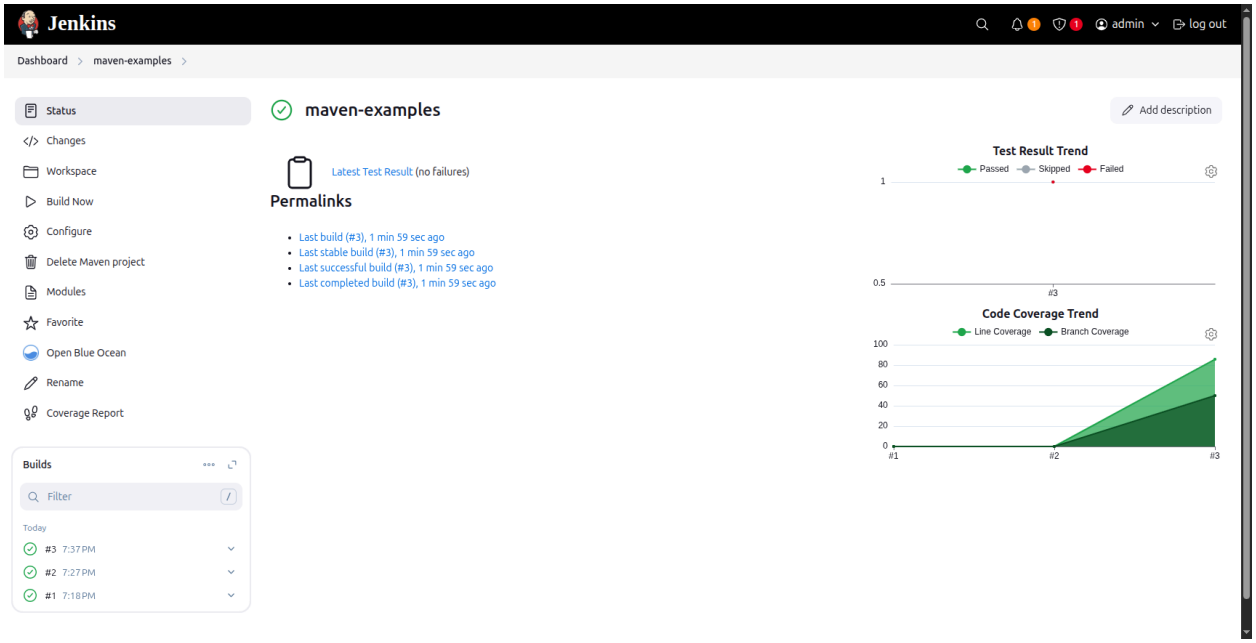
Select the `ut1` branch as the branch to **compare** and click **Create pull request**:



Approve the request as normal.

This will add pre-written tests to your master branch so that Jacoco will show some coverage in Jenkins.

Go back to the Maven project page you created earlier and build the project; refresh the page when the build is finished and you should see a code coverage report get published on the project page.



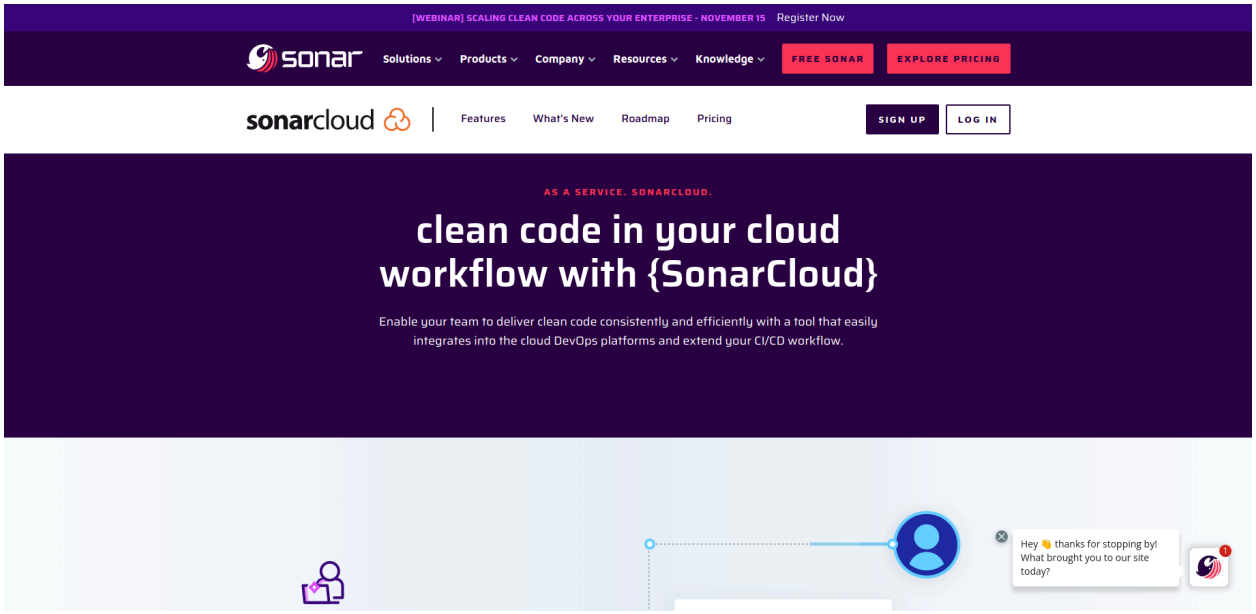
Adding Static Code Analysis with SonarQube

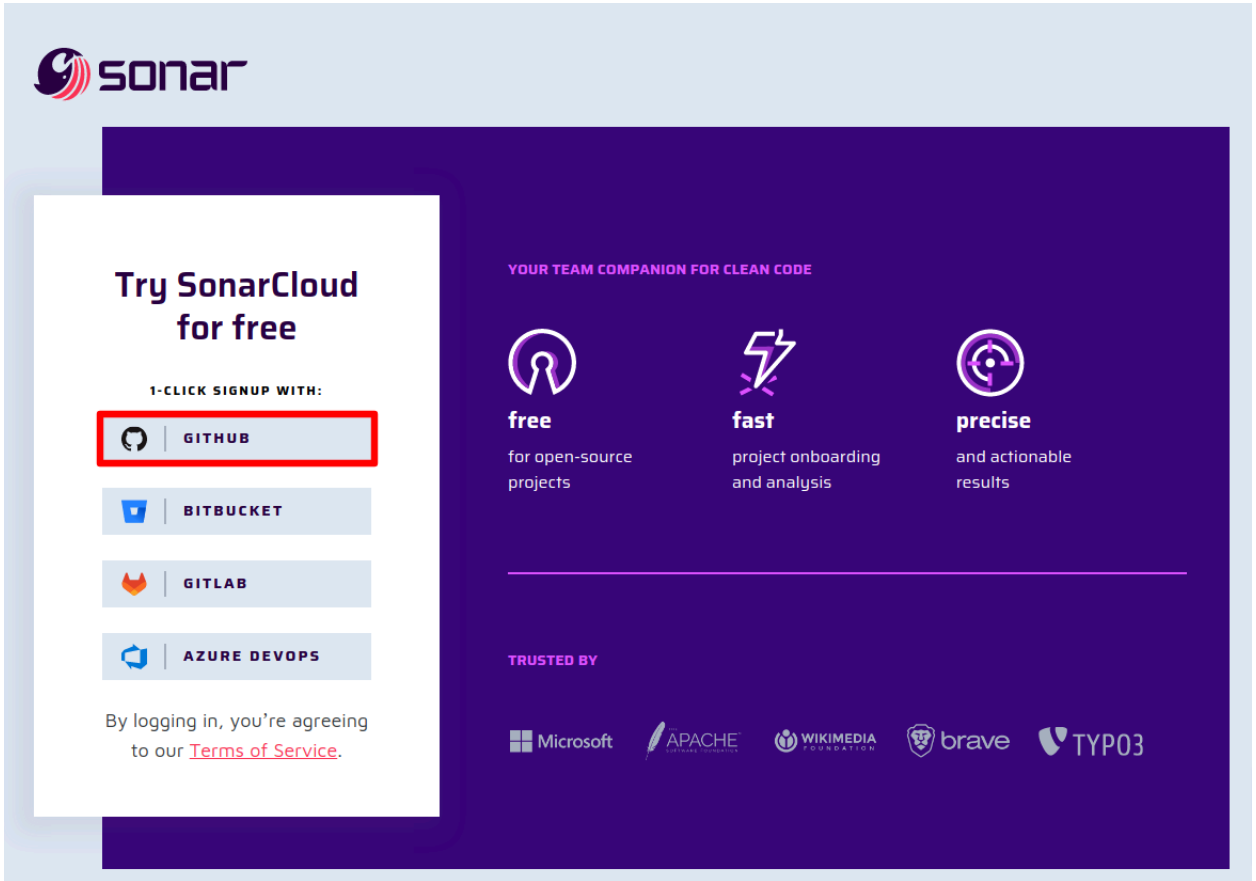
Steps:

- Set up Sonar Cloud.
- Install SonarQube Scanner plugin for Jenkins.
- Configure SonarQube Servers from Jenkins Systems Configuration.

Set Up Sonar Cloud as SonarQube Server

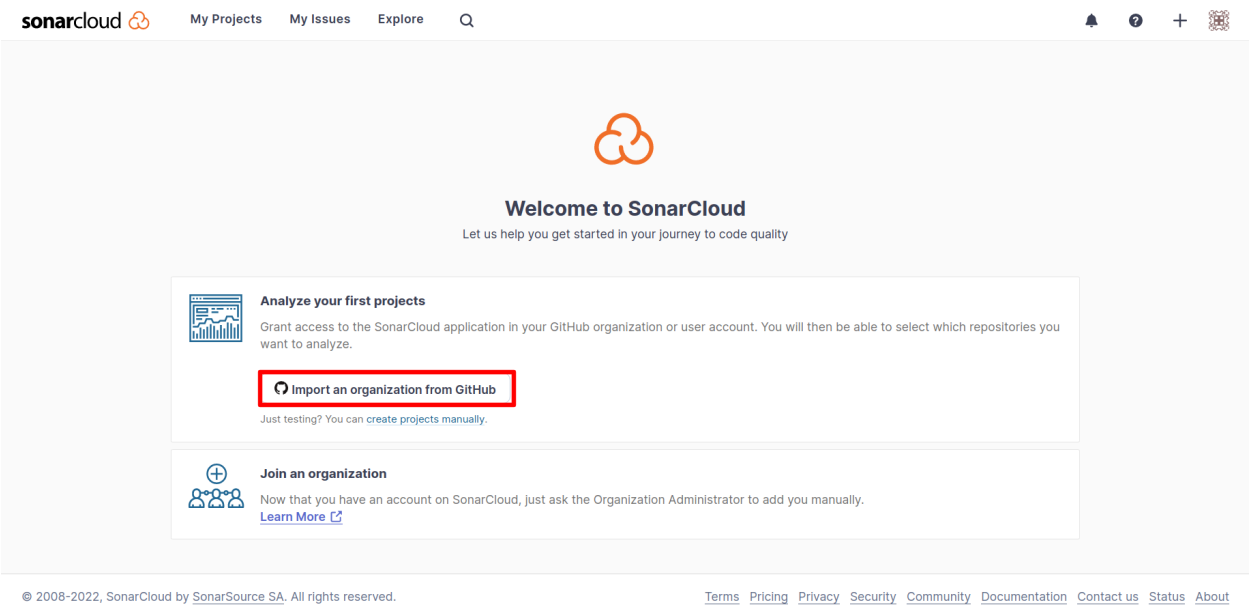
Head over to SonarCloud and sign up using your GitHub account.



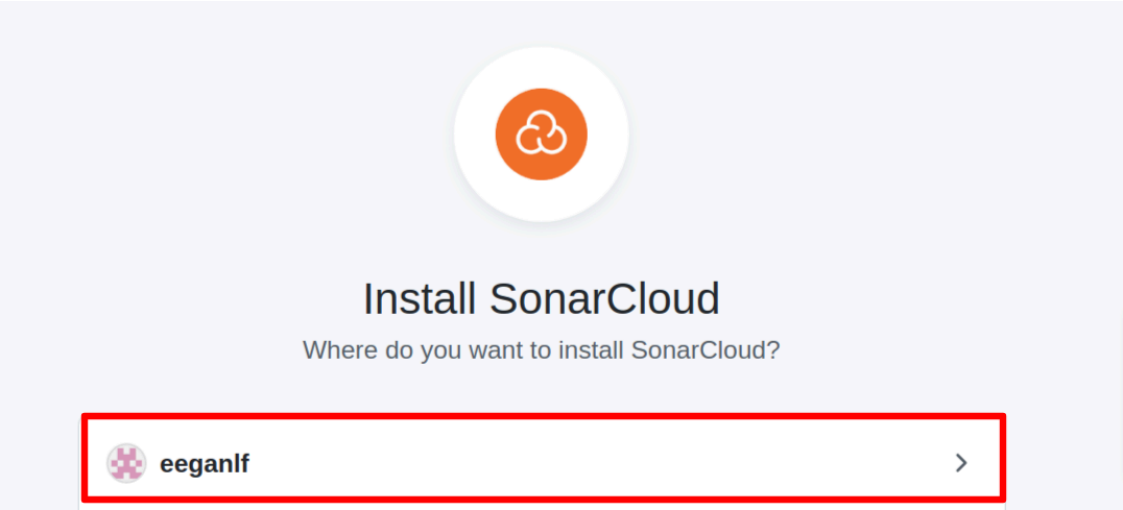


Ensure that you are logged into the GitHub account from the same browser in order to provide authorization. Follow the prompts to authorize SonarCloud with GitHub.

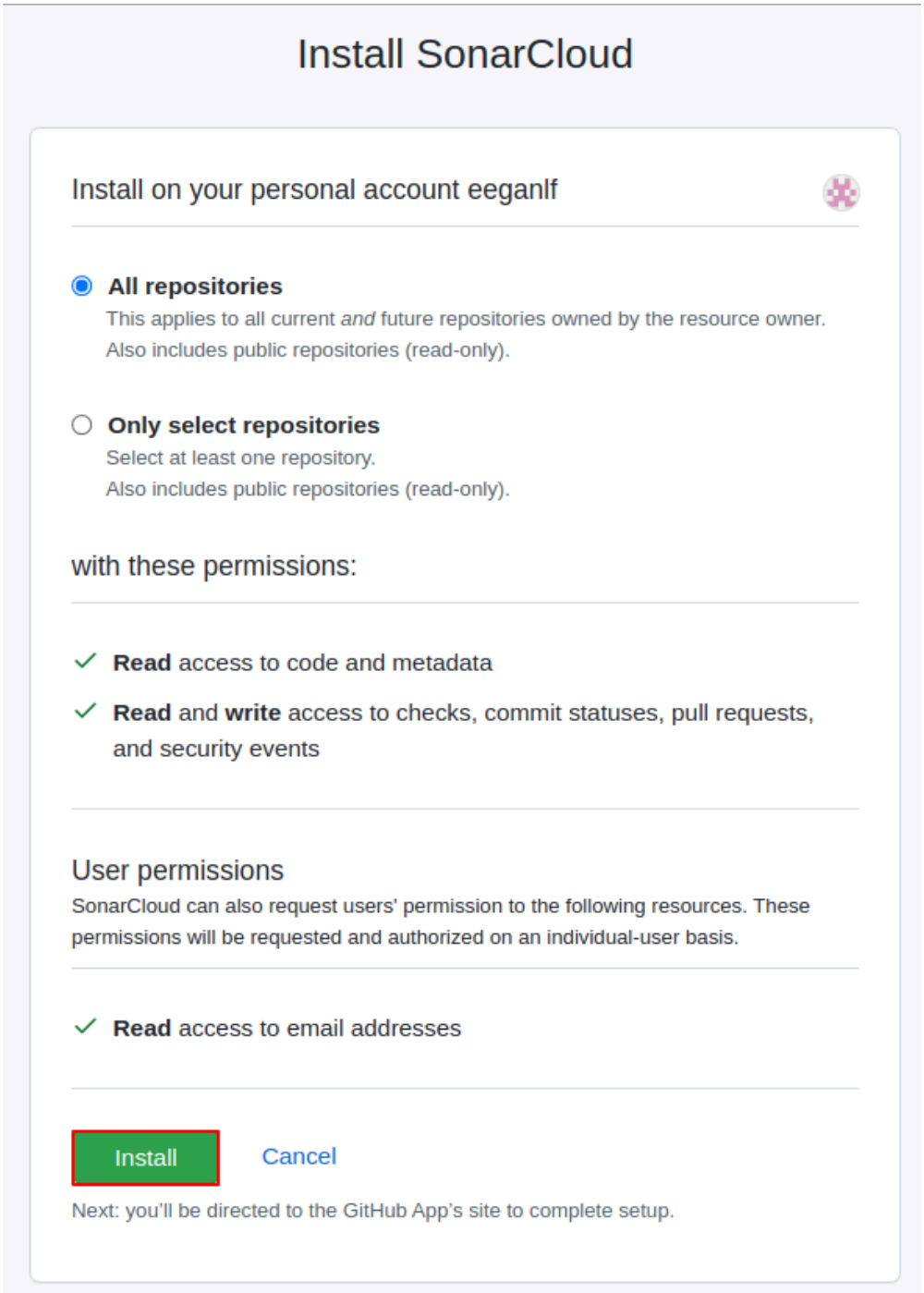
From the welcome page, click **Import an organization from GitHub.**



Choose your GitHub organization.

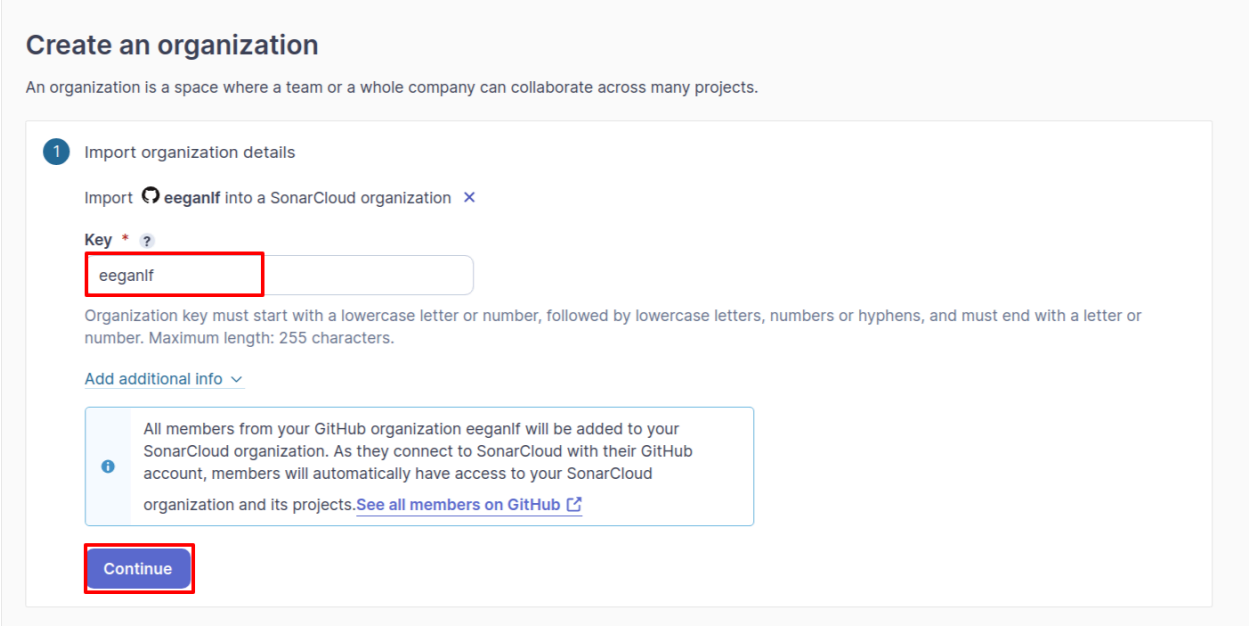


Provide access to all your repositories:

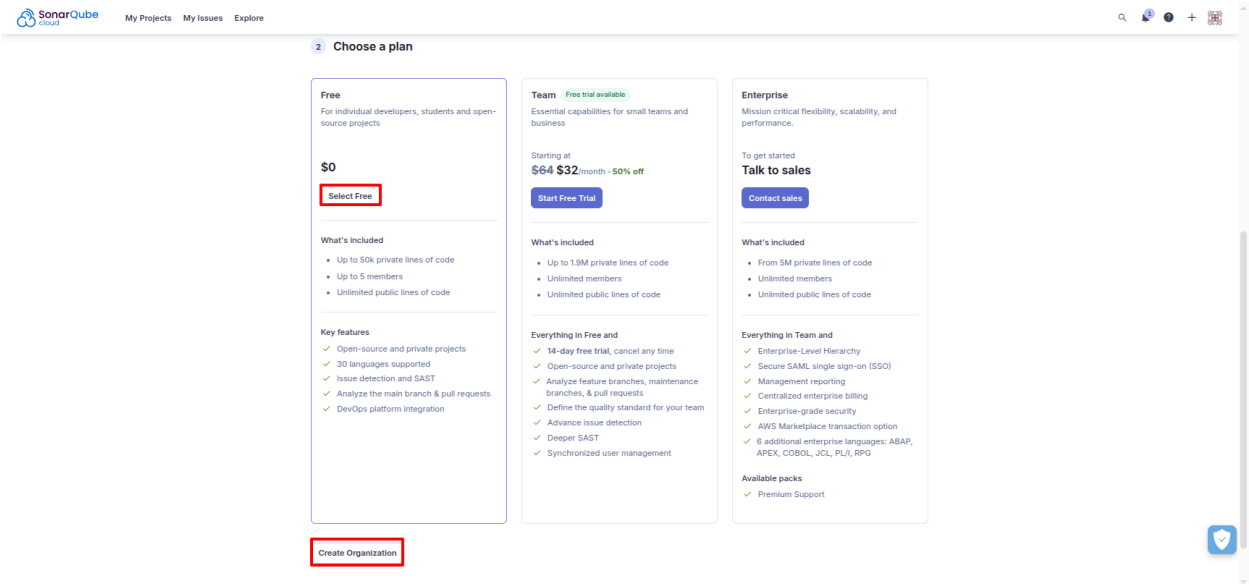


You will be redirected back to SonarCloud.

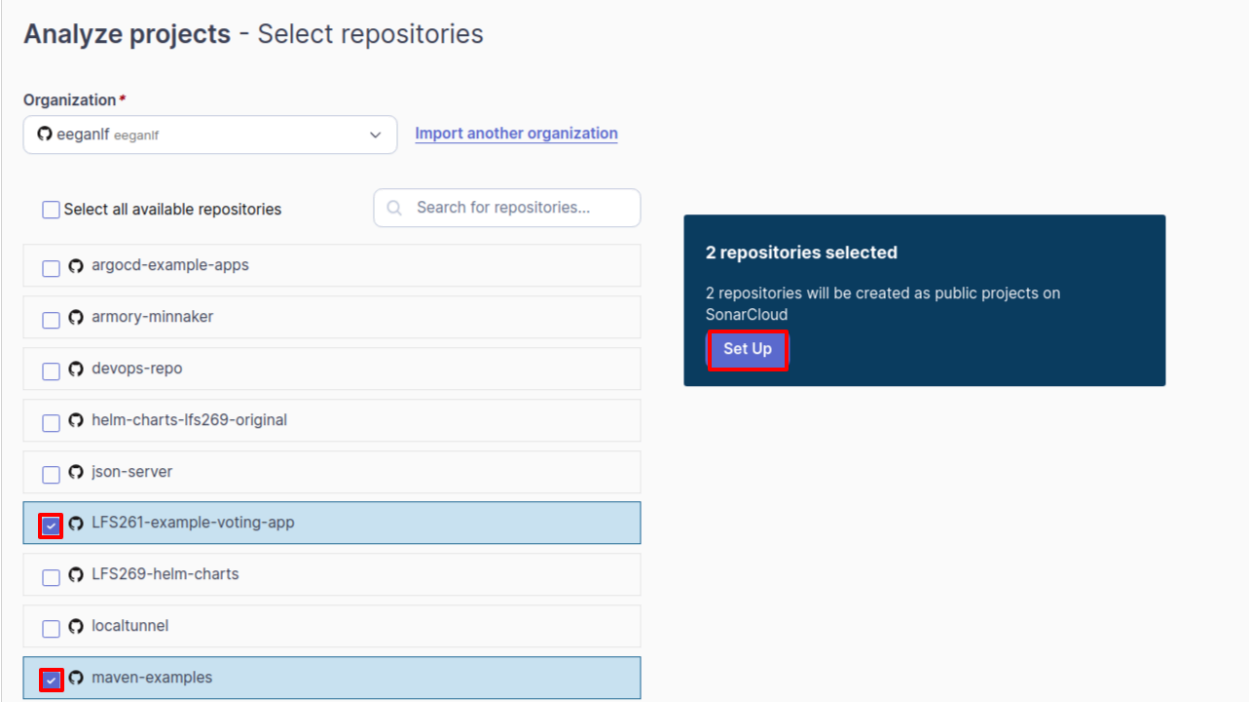
Provide an organization name to be created on SonarCloud. This is important, as you are going to use it as a reference later when you integrate it with Jenkins. Your GitHub username should be auto-populated. If not, enter it into the **Key** box. Click **Continue**.



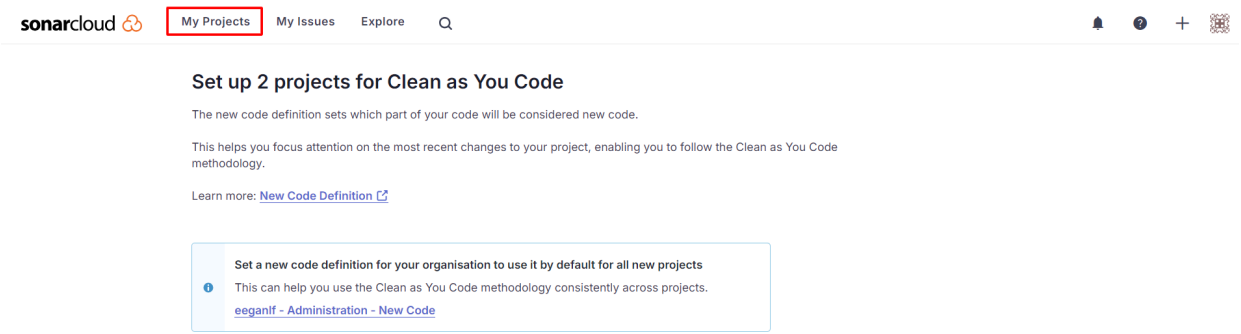
Since your repository is public, you can proceed with the free plan.



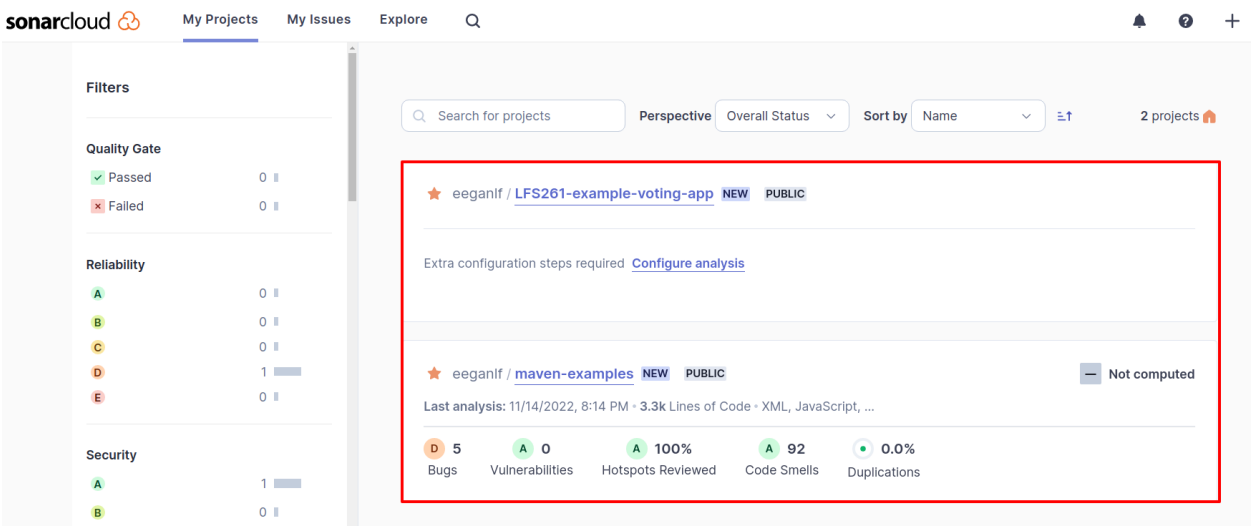
Choose the repositories that you want to analyze with SonarQube. Select `maven-examples` and the `example-voting-app`. Click **Set Up**:



If you are not redirected to your **My Projects** page in SonarCloud, navigate there now via the **My Projects** link.



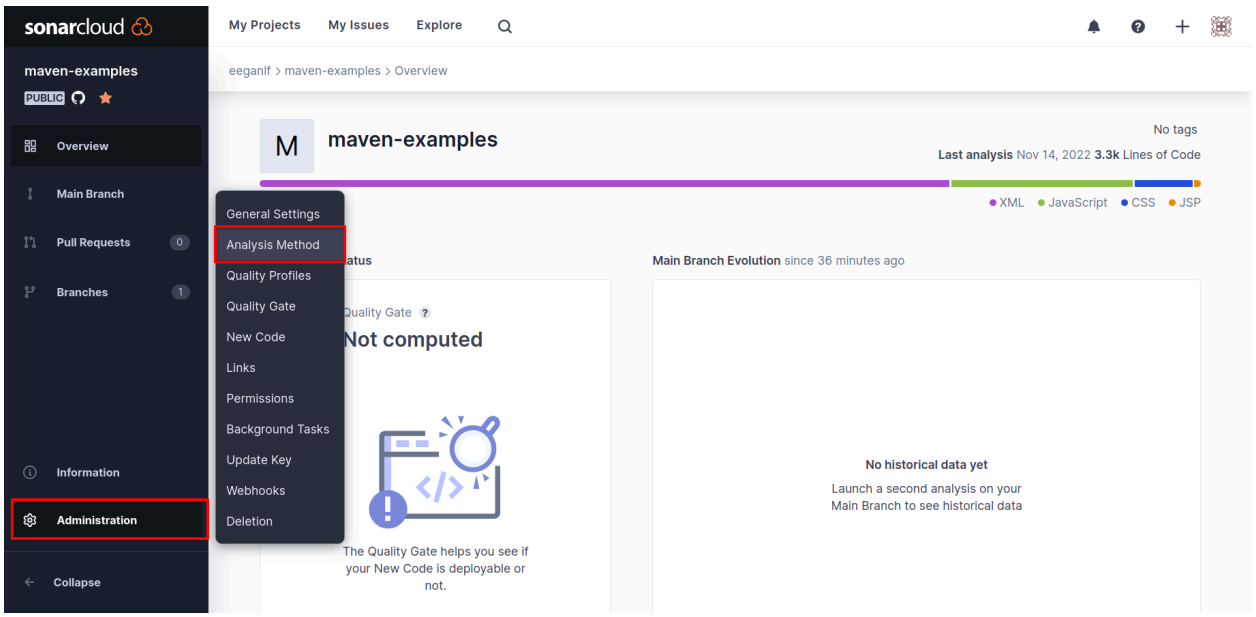
This is the page that lists all of your projects.



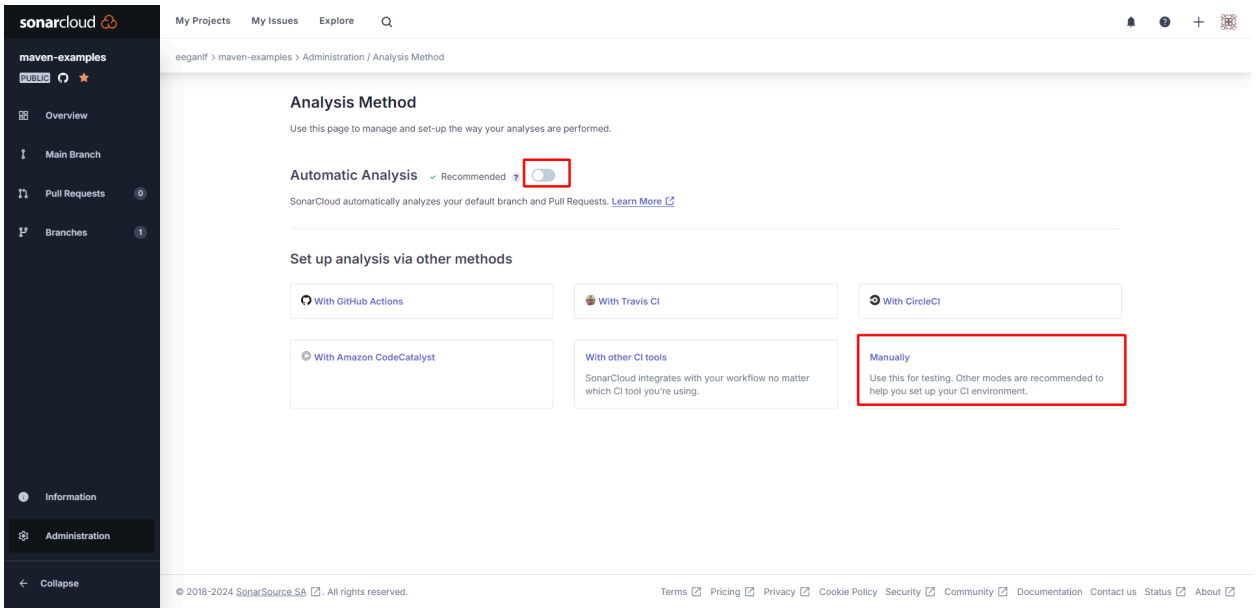
Click on the **maven-examples** project:

You will be taken to the **maven-examples** project page. You must disable automatic analysis as you want Jenkins to tell SonarCloud when to analyze your code.

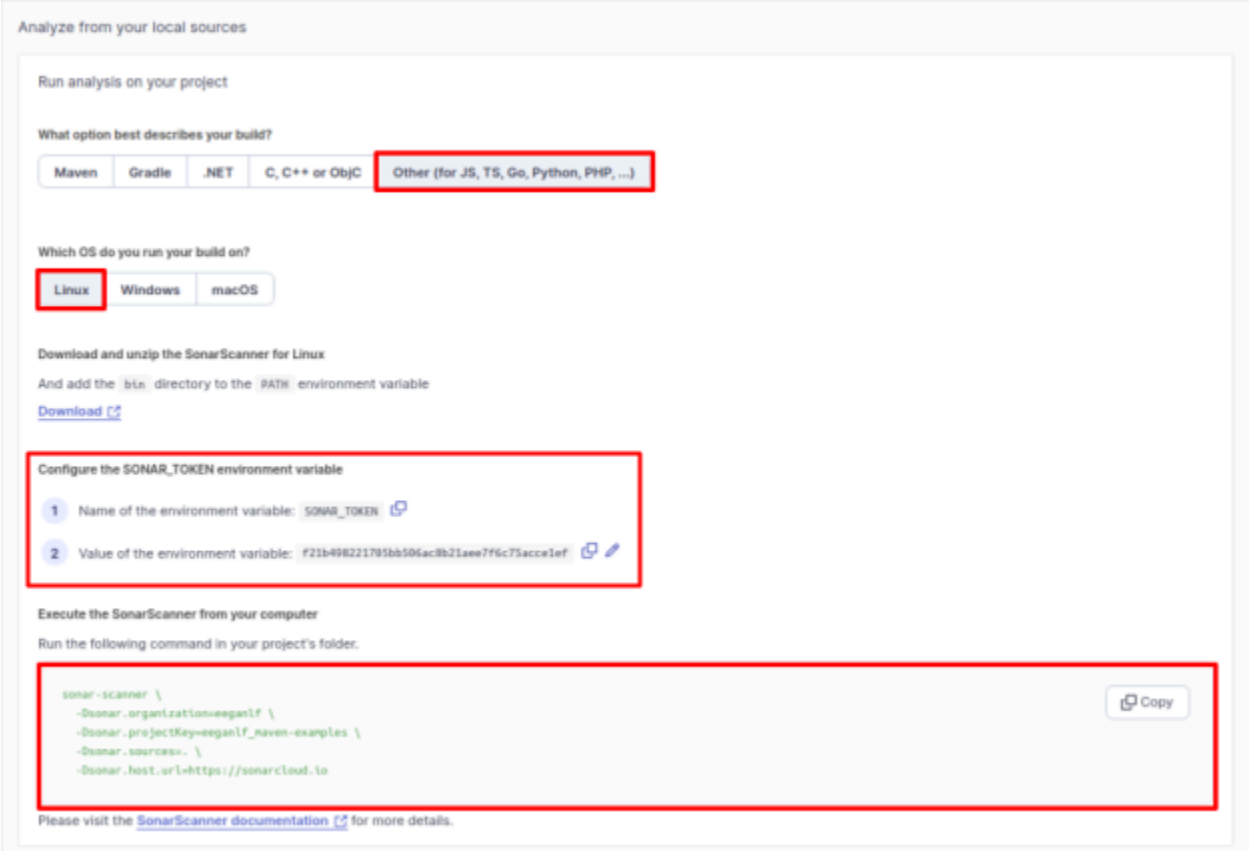
You must also configure your project. To accomplish this, click navigate to **Administration > Analysis Method.**



Uncheck **Automatic Analysis** and click **Set up analysis via other methods > Manually**:



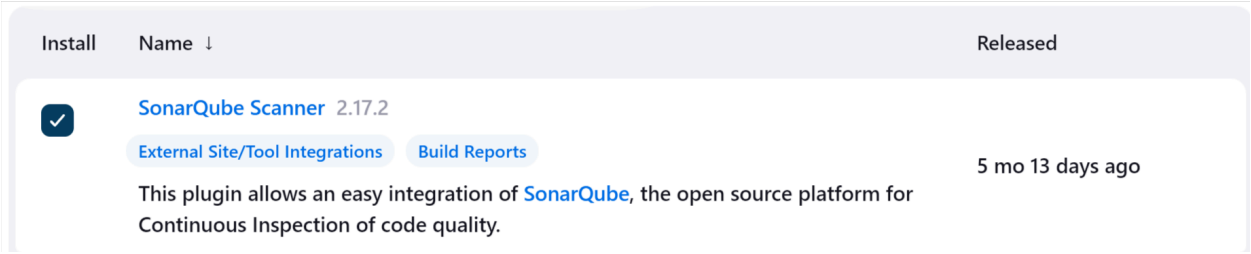
Under **What option best describes your build?** select **Other**. Select **Linux** for the operating system.



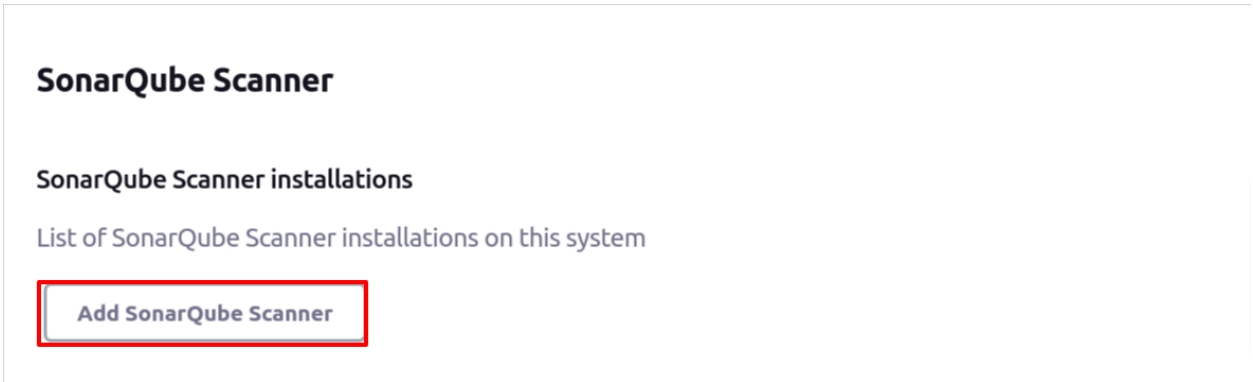
Notice that a command is generated. This command contains configuration properties that we will use to integrate SonarQube with Jenkins.

Integrate SonarQube with Jenkins

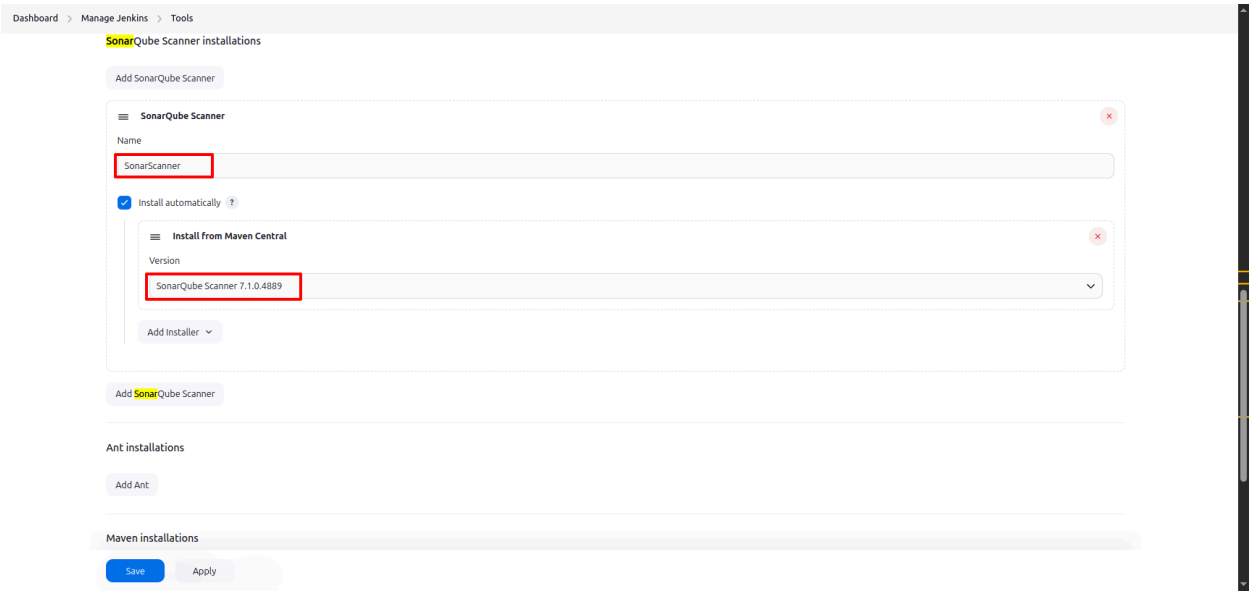
Install the SonarQube Scanner plugin for Jenkins.



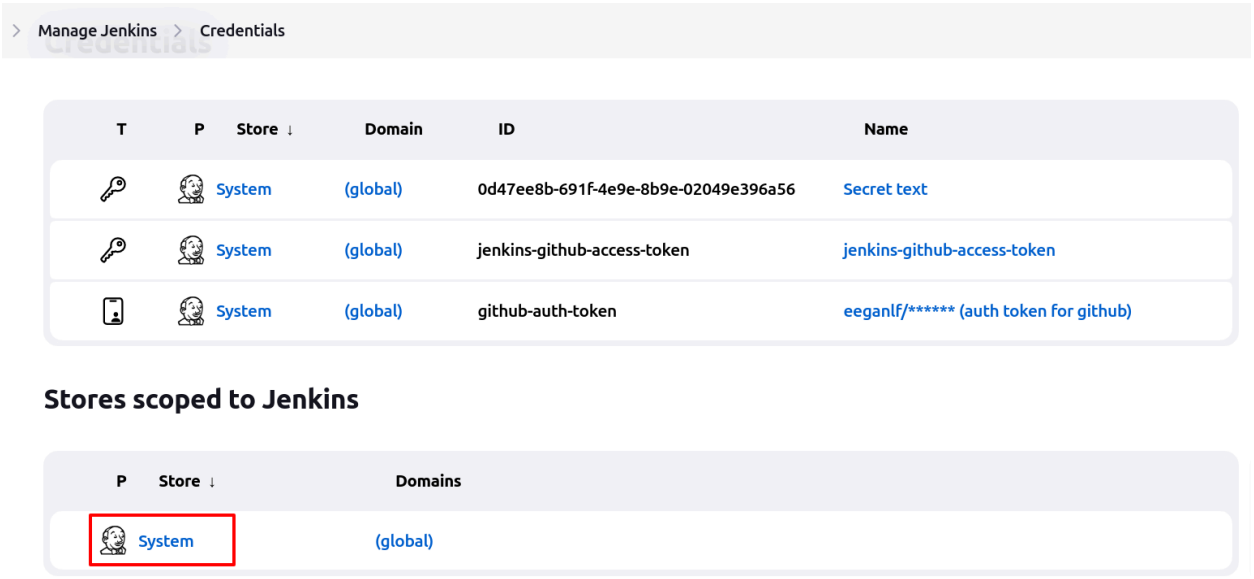
From **Jenkins > Manage Jenkins > Tools > SonarQube Scanner** click **Add SonarQube Scanner**:



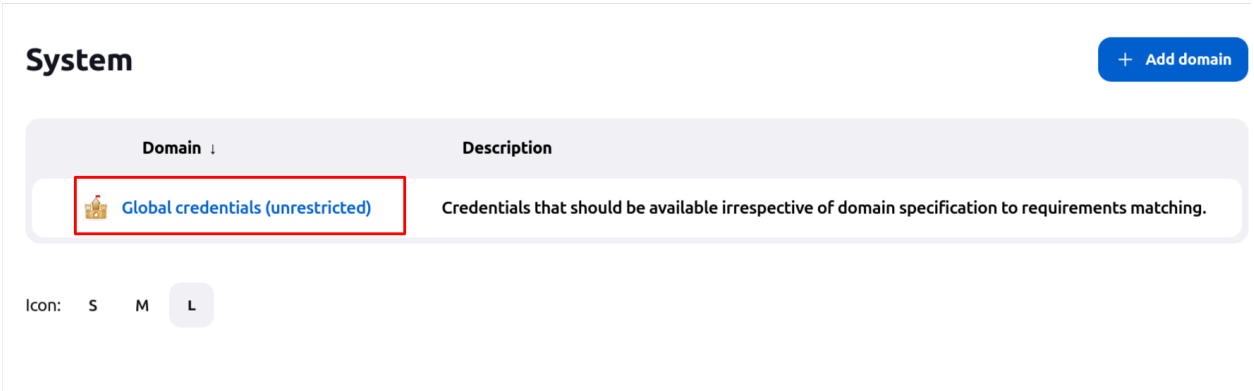
Name the SonarQube Scanner **SonarScanner** which will be referred to later when you write the pipeline code. Also, choose whichever is the latest version, which should be automatically selected from the dropdown menu:



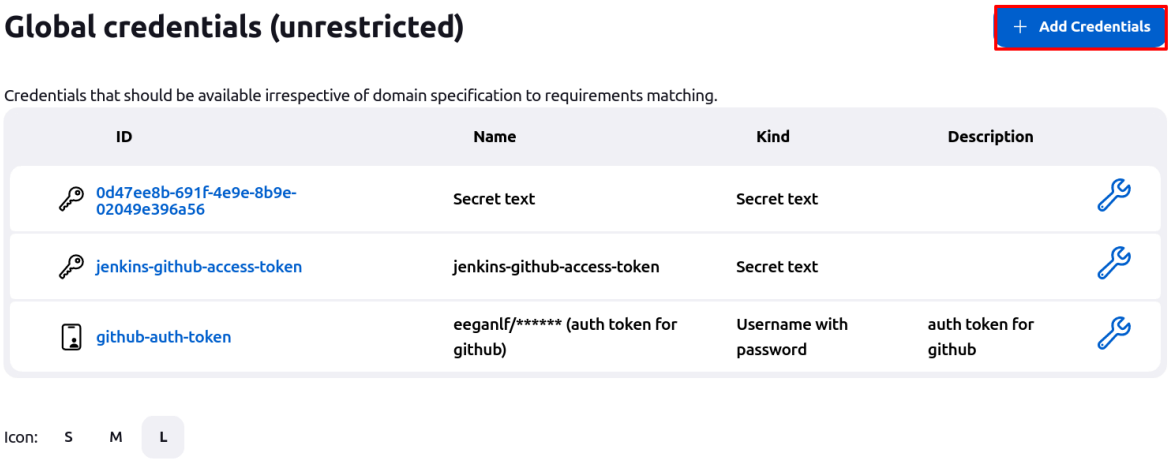
From the Jenkins home, navigate to **Manage Jenkins > Security > Credentials**:



Click **Global credentials (unrestricted)**:

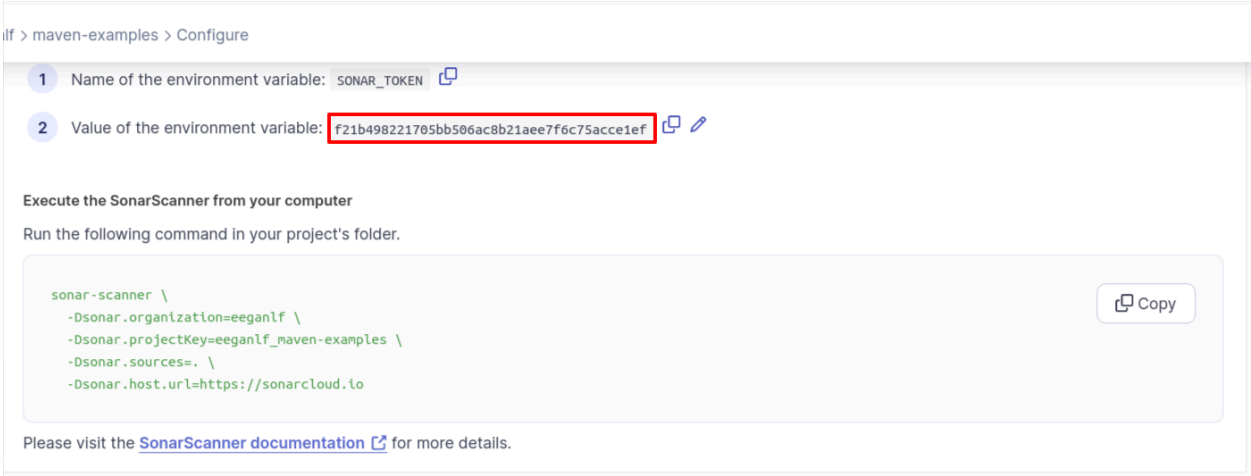


Click **Add Credentials**:



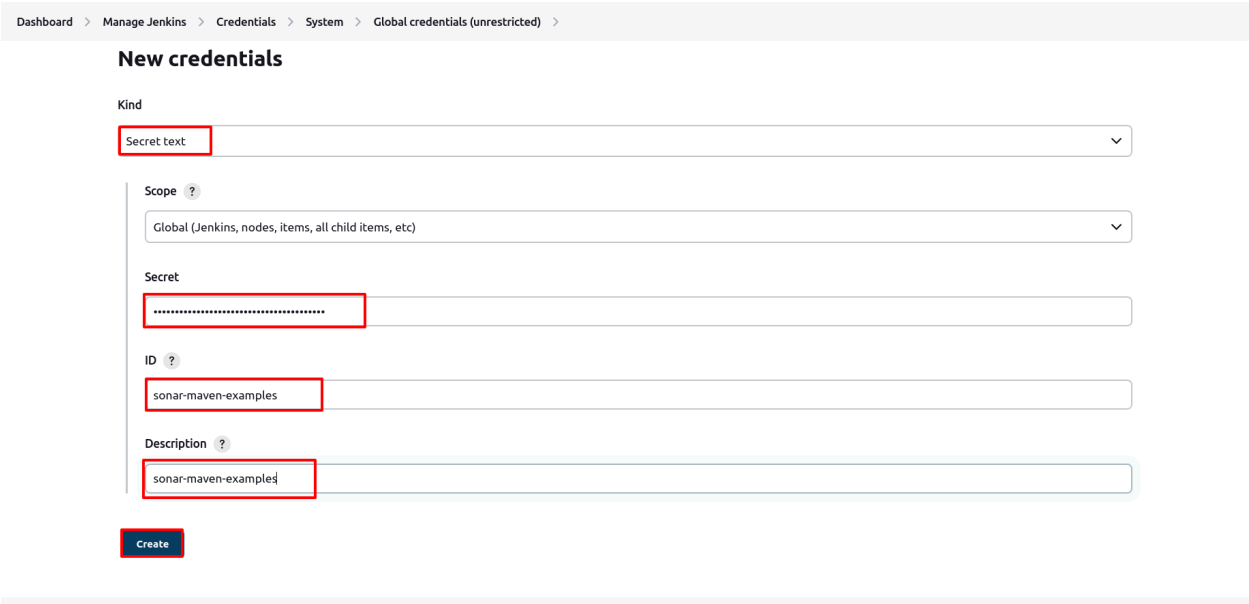
Add new credentials with:

- Kind “Secret text”.
- Copy the value for the **`SONAR_TOKEN`** found on the configuration page on SonarCloud for the **`maven-examples`** project:

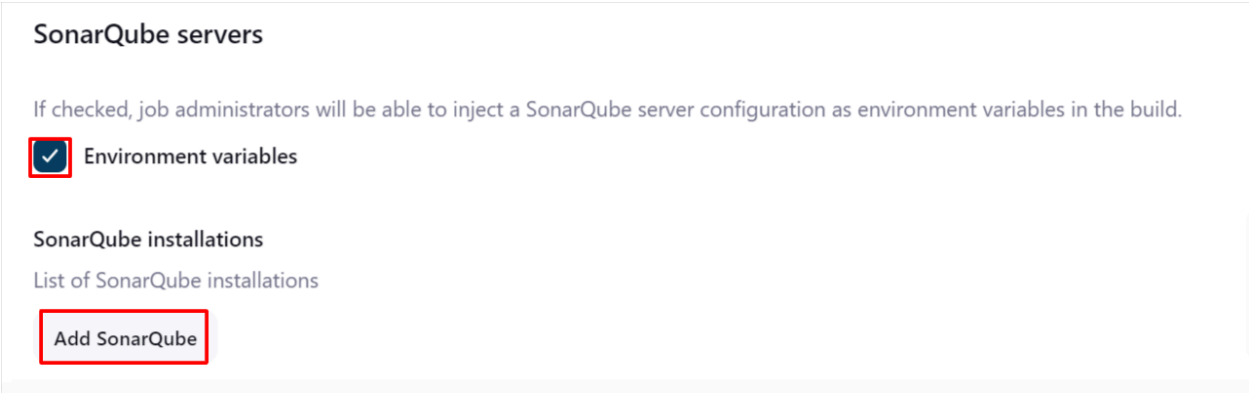


- Paste it in the “Secret” field on Jenkins.
- Add ID and Description as **`sonar-maven-examples`**.

Refer to the following screenshot while adding credentials:



Now navigate to **Jenkins > Manage Jenkins > System > SonarQube Servers** and add a new entry.

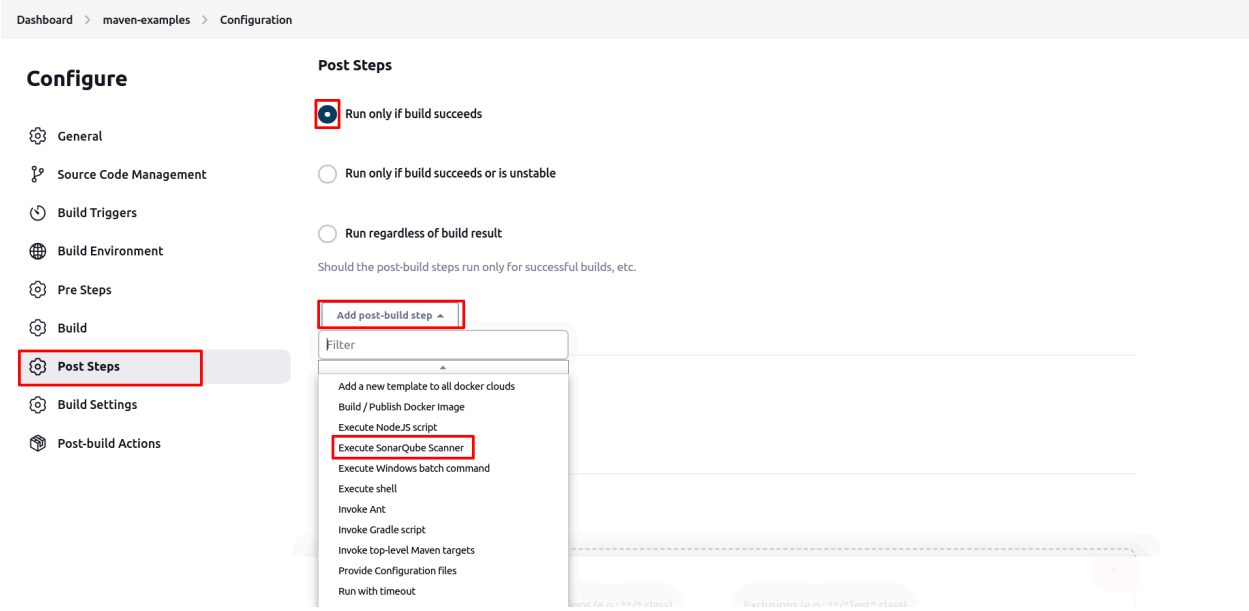


Provide the configuration as shown below:

- Check the **Environment Variables** box to enable injection of configurations.
- Name this instance **`sonar-maven-examples`** as it will be referred to later.
- Use the SonarQube URL but do **NOT** include a trailing slash or it will not work.
- Select the credentials you added in the previous step from the dropdown, i.e. *sonar-maven-examples*.
- Save the configuration.

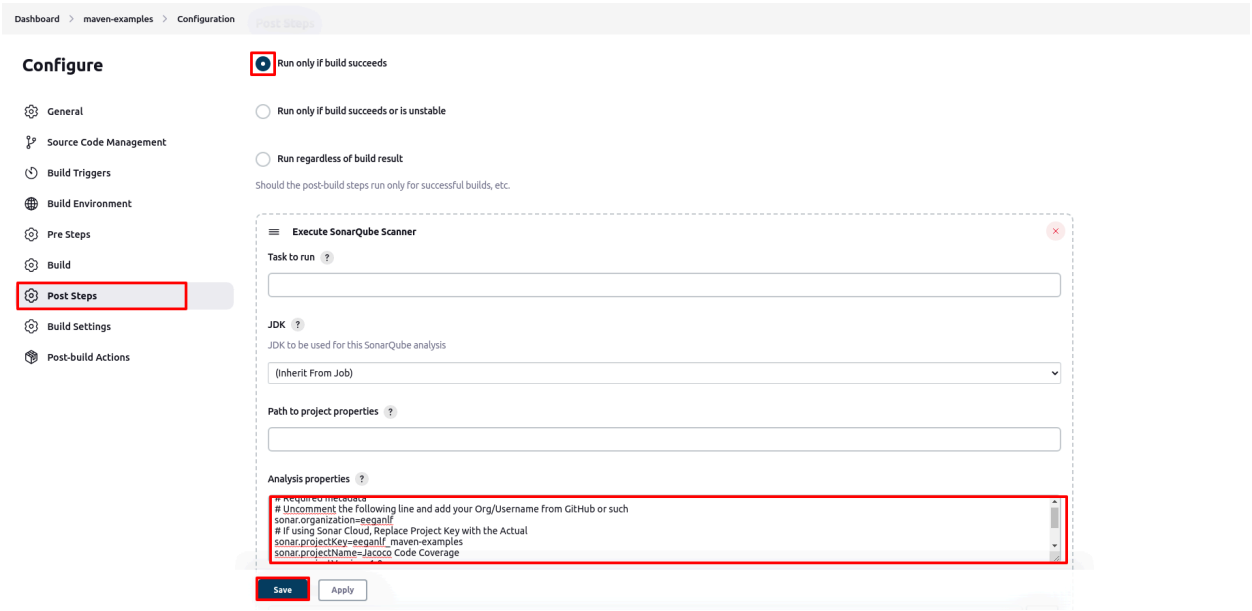
Scanning Code and Reporting to SonarQube

In Jenkins, go to the configuration page for the **`maven-examples`** job that you created to use code coverage and add a post build step. Select **Execute Sonarqube Scanner**.

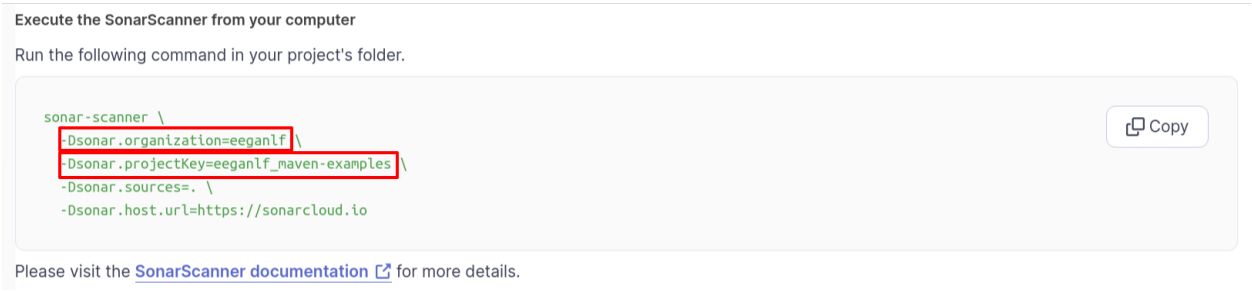


Put the following snippet into the **Analysis properties** text box:

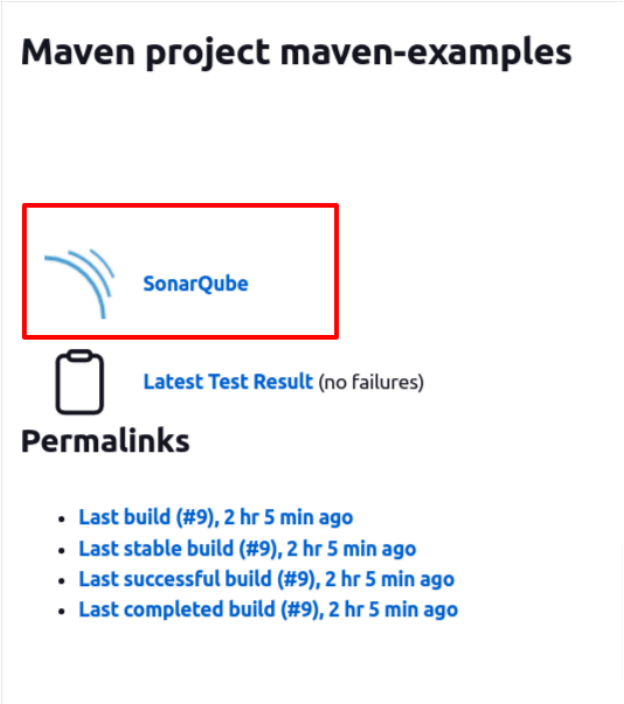
```
# Required metadata
# Uncomment the following line and add your Org/Username from GitHub
or such
sonar.organization=YourGitHubOrg
# If using Sonar Cloud, Replace Project Key with the Actual

sonar.projectKey=yourGitHubOrg_ProjectName
sonar.projectName=Jacoco Code Coverage
sonar.projectVersion=1.0
# Comma-separated paths to directories with sources (required)
sonar.sources=code-coverage-jacoco/src/main
# Encoding of the source files
sonar.sourceEncoding=UTF-8
sonar.java.binaries=.
sonar.coverage.jacoco.xmlReportPaths=code-coverage-jacoco/target/site/
jacoco-ut/jacoco.xml
```



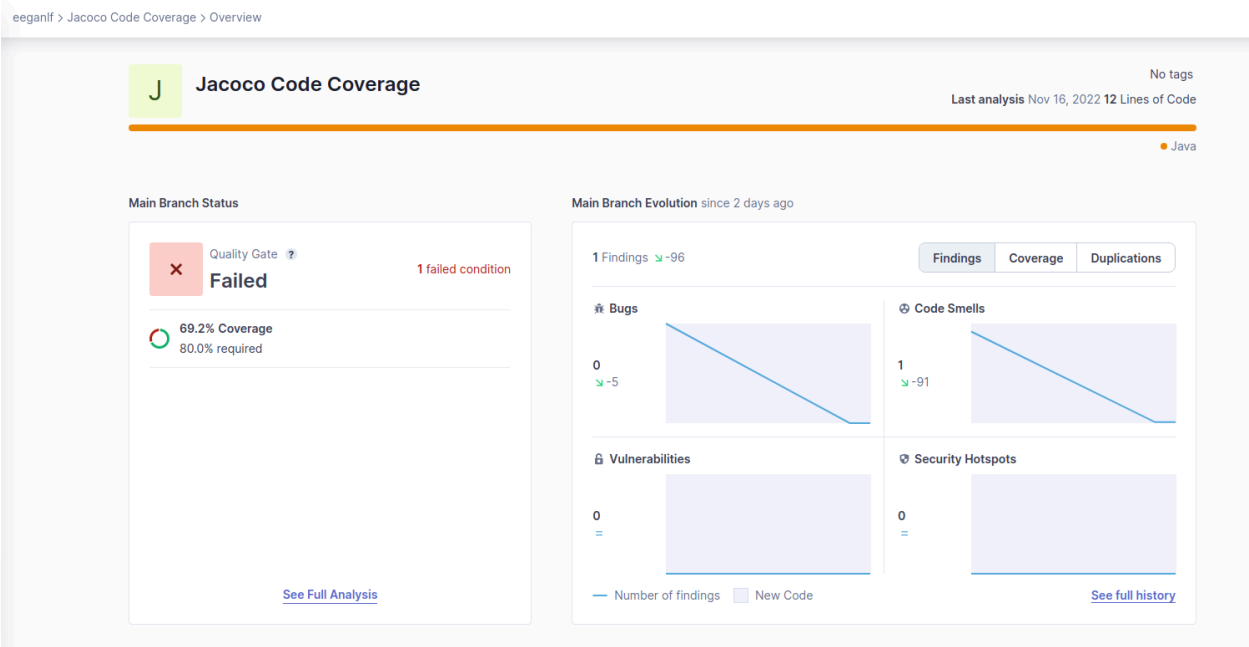
Ensure you replace the properties for **`sonar.organization`** and **`sonar.projectKey`** with what you see in the **Sonar Cloud Analysis Configuration** page:



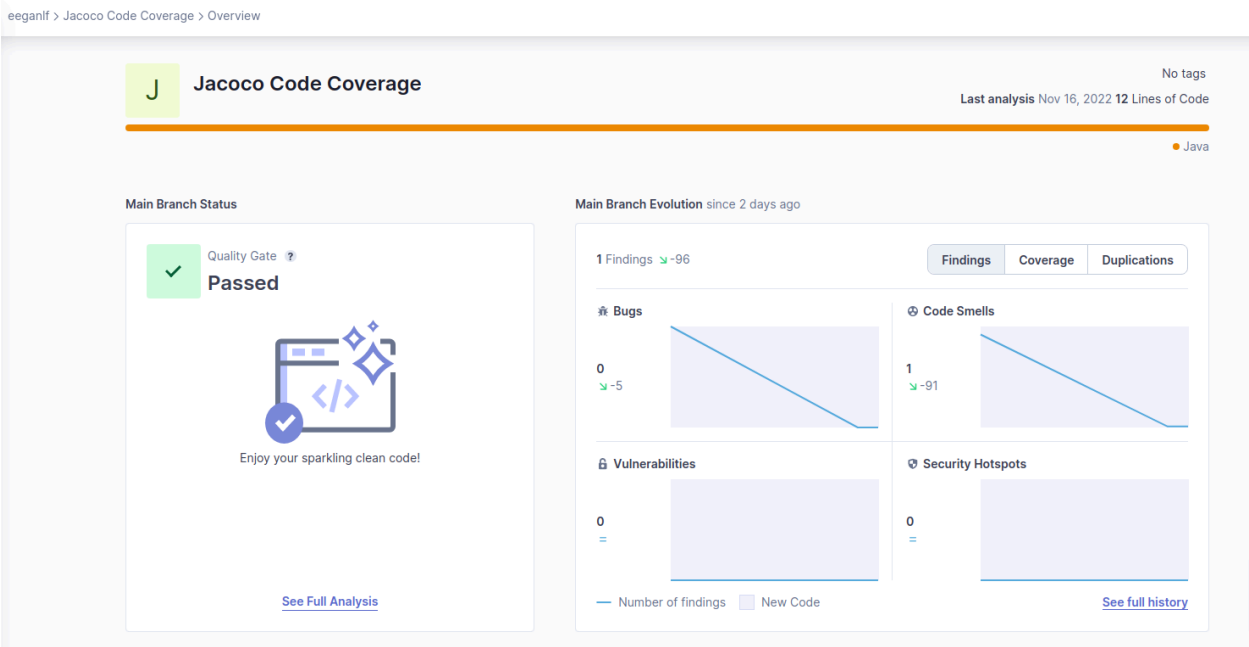
Back in Jenkins, build the project. Click the SonarQube link to navigate to SonarQube:



You will see that sonarcloud.io shows that the project failed and displays the reason why it failed. In this case, it failed because we have less than 80% test coverage.



Back on the GitHub repository for **`maven-examples`**, merge the **`ut2`** branch into the master and run **Build Now** in Jenkins on the **`maven-examples`** job. After it is complete, follow the **SonarQube** link. The job should now be displayed as passed:



Note that this does not keep Jenkins from successfully building the job. It is an illustration of how quality gates can be configured to output a pass or fail result for a particular job. We do, however, want to enable SonarQube to fail builds before they get to production. We can and should do this in the Jenkinsfile. We will accomplish this in the next section for the **`example-voting-app`**.

Adding SonarQube Scanner Stage to Jenkinsfile

You want SonarQube to fail the pipeline before the project is deployed if the code does not pass analysis. Add the following code snippet to the consolidated Jenkinsfile created earlier while setting up a monopipe. Place the code above the **`stage ('deploy to dev')`** block of code.

`file: example-voting-app/Jenkinsfile`

```
    stage('Sonarqube') {
      agent any
      when{
        branch 'master'
      }
            environment{
```

```
        sonarpath = tool 'SonarScanner'
      }

      steps {
            echo 'Running Sonarqube Analysis..'
            withSonarQubeEnv('sonar-instavote') {
              sh "${sonarpath}/bin/sonar-scanner
-Dproject.settings=sonar-project.properties
-Dorg.jenkinsci.plugins.durabletask.BourneShellScript.HEARTBEAT_CHECK_
INTERVAL=86400"
            }
      }
    }


    stage("Quality Gate") {
        steps {
            timeout(time: 1, unit: 'HOURS') {
                // Parameter indicates whether to set pipeline to
UNSTABLE if Quality Gate fails
                // true = set pipeline to UNSTABLE, false = don't
                waitForQualityGate abortPipeline: true
            }
        }
    }
```

You can access this snippet here.

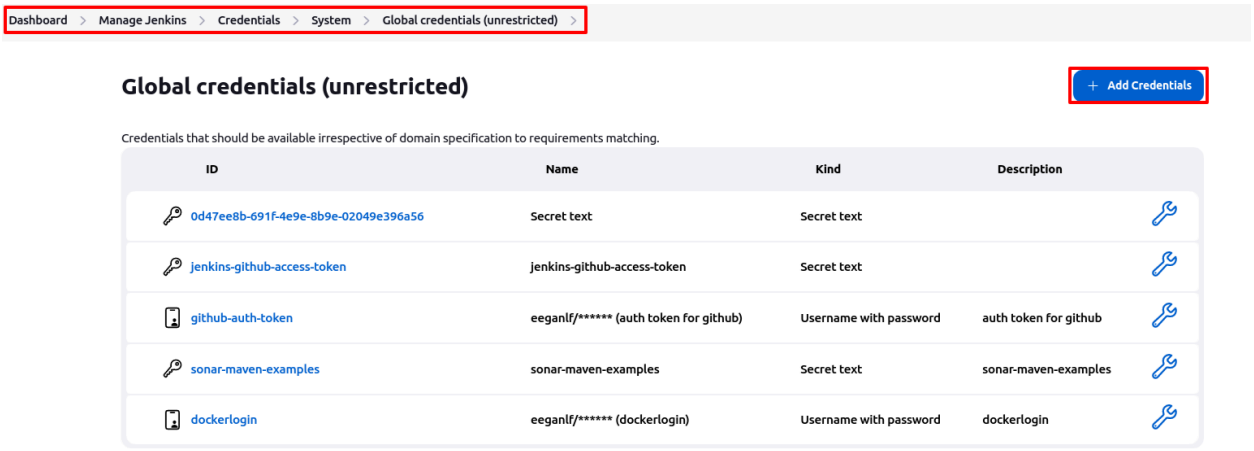
Also update `sonar-project.properties` file at the root of `example-voting-app` repository with *your* organization and project key as per configured on SonarCloud.

File: `example-voting-app/sonar-project.properties`

```
# Uncomment and update Org matching your configurations on Sonarcloud
sonar.organization=your-org
sonar.projectKey=your-org_example-voting-app
sonar.projectName=LFS261 example voting app
sonar.projectVersion=1.0

# Comma-separated paths to directories with sources (required)

sonar.sources=worker

# Encoding of the source files

sonar.sourceEncoding=UTF-8
```

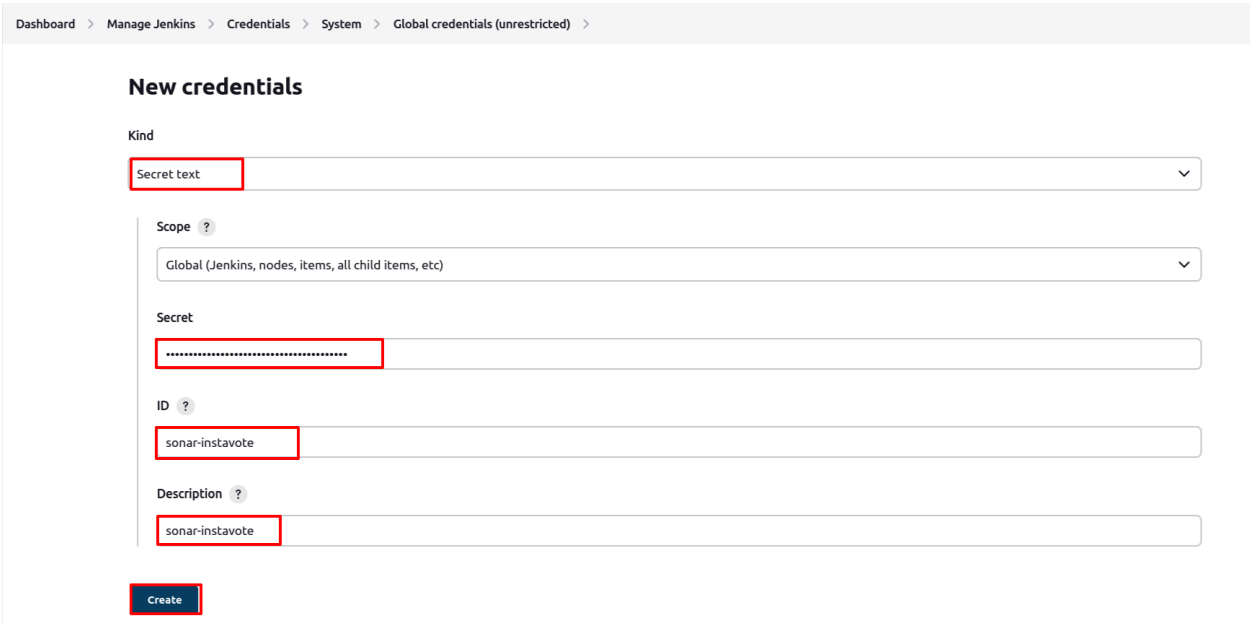
```
sonar.java.binaries=worker

sonar.coverage.jacoco.xmlReportPaths=worker/target
```

You also need to add Credentials to connect to the project added to SonarCloud. Navigate to **Jenkins > Manage Jenkins > Credentials > System > Global credentials (unrestricted)** and click **Add Credentials**:

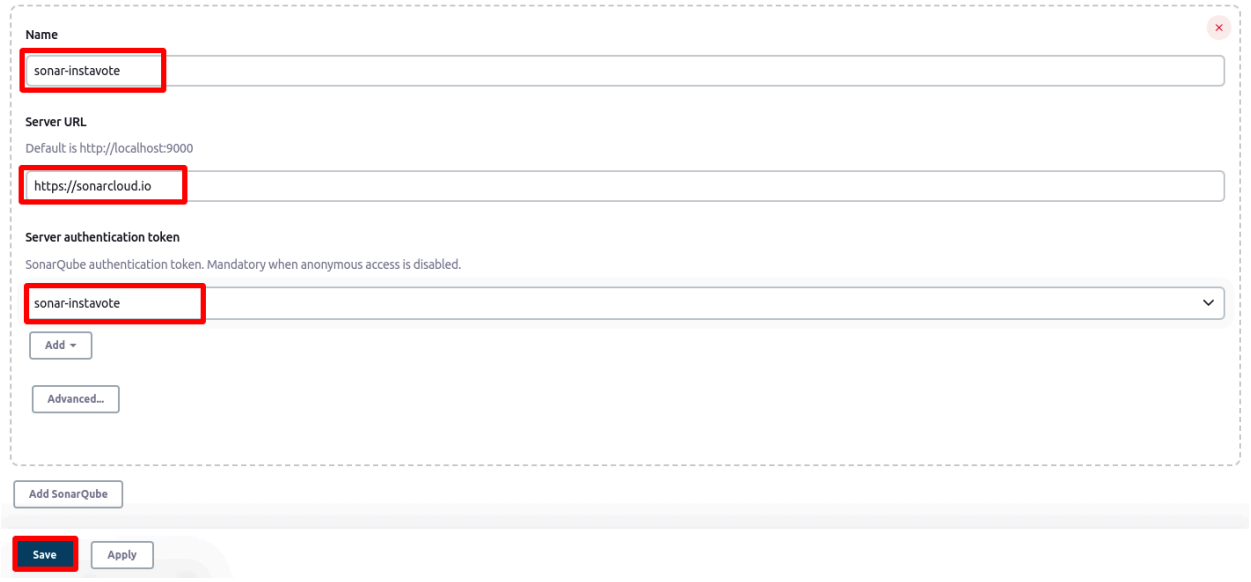


Find the **SONAR_TOKEN** on the **LFS261-example-voting-app** configuration page and copy it:

Paste this token into the **Secret** text box while configuring the credentials:



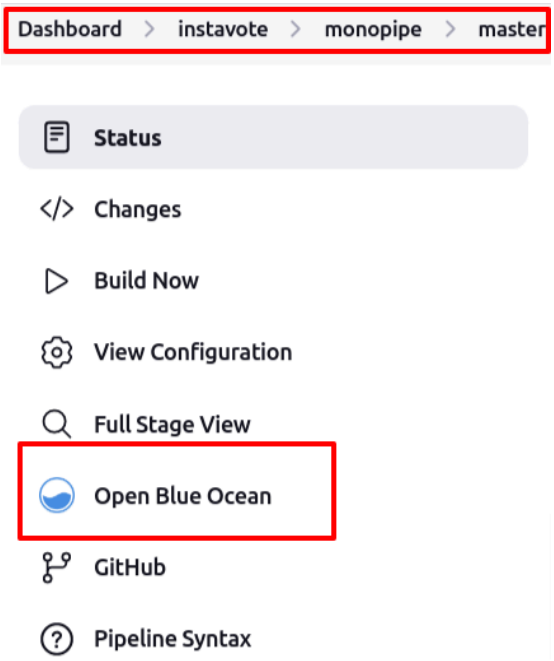
Create a new SonarQube Server configuration from **Jenkins > Manage Jenkins > System > SonarQube Servers**.



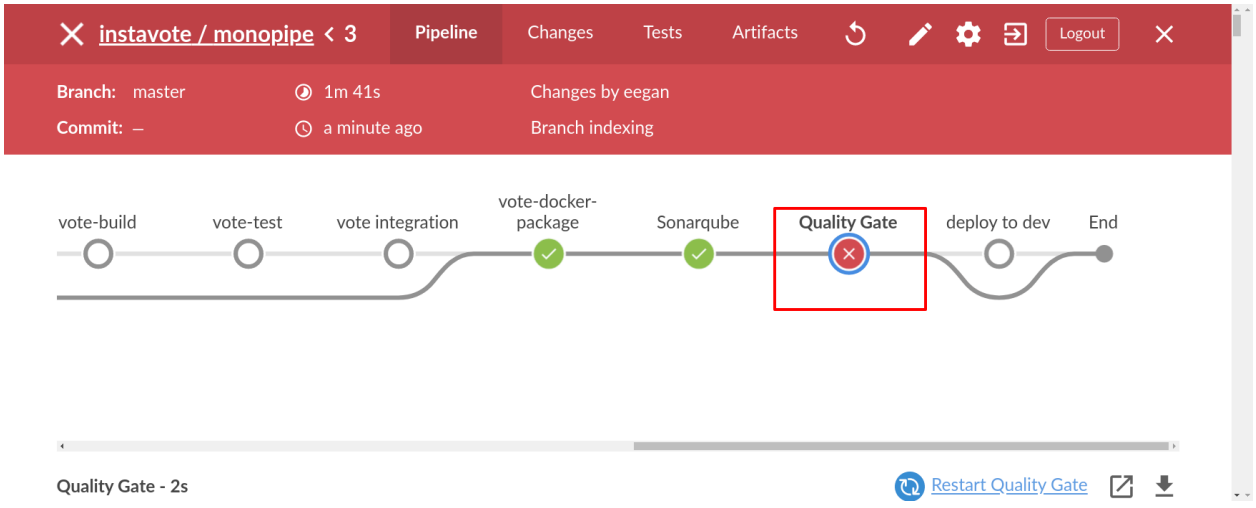
Remember to make sure that the **`https://sonarcloud.io`** does **NOT** have a trailing slash.

The free quality gate provided by SonarCloud only performs analysis on 20 lines or more of new code. Replace the entirety of the **`example-voting-app/worker/src/main/java/worker/Worker.java`** file with this code.

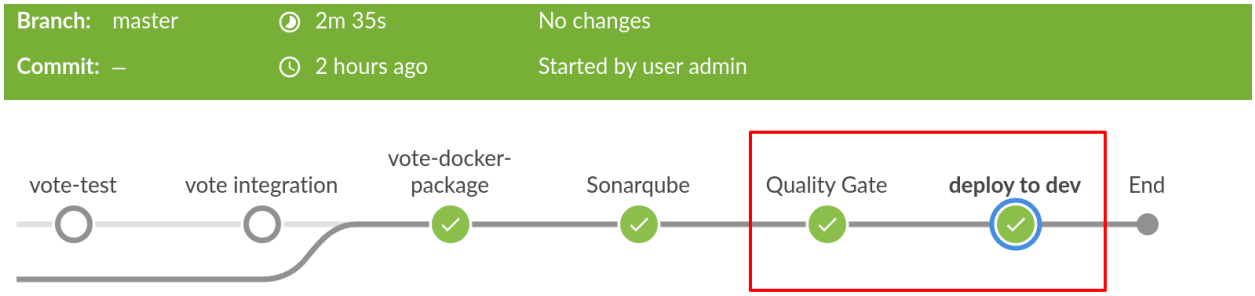
Commit to the master branch and observe the latest jobs being run with the next instance of the pipeline run. To view the build in the alternate BlueOcean interface, select the interface from the side menu of the master branch:



After the master branch builds in the monopipe job, you will notice that it fails at the **Quality Gate** step:



To get our worker app to pass the quality gate, we need to provide test coverage greater than 80%. Replace `LFS261-example-voting-app/worker/src/test/java/worker/UnitWorker.java` with this code.



Adding Integration Tests

Integration tests for the **vote** Python app are already added to the `vote/` subdirectory of the repository. To add integration tests to the pipeline, follow the steps below.

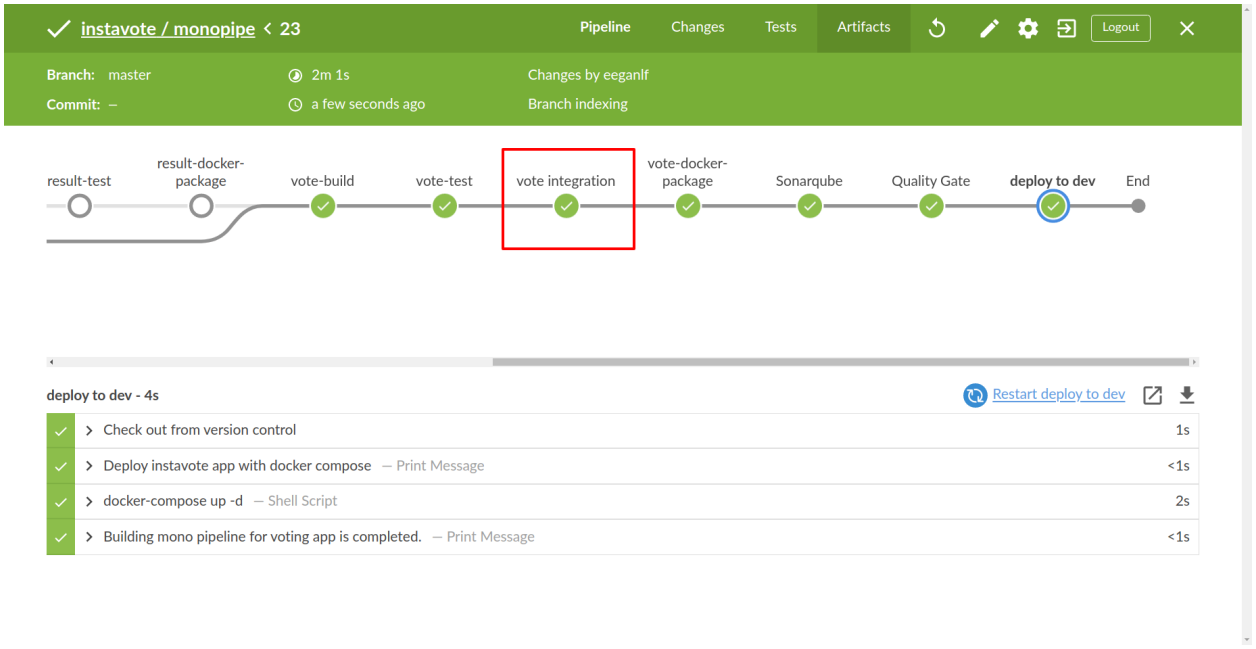
Examine the script at `vote/integration_test.sh` in the `example-voting-app` repository. Reading the script and the docker compose setup it uses should give you a good understanding of how integration tests are set up. Add a stage to the Jenkinsfile to run the integration tests as follows. Place the below code block just after the stage **vote-test** stage which runs unit tests for the **vote** app:

`file: example-voting-app/Jenkinsfile`

```
stage('vote integration'){
    agent any
    when{
      changeset "**/vote/**"
      branch 'master'
    }
    steps{
      echo 'Running Integration Tests on vote app'
      dir('vote'){
        sh 'sh integration_test.sh'
      }
    }
```

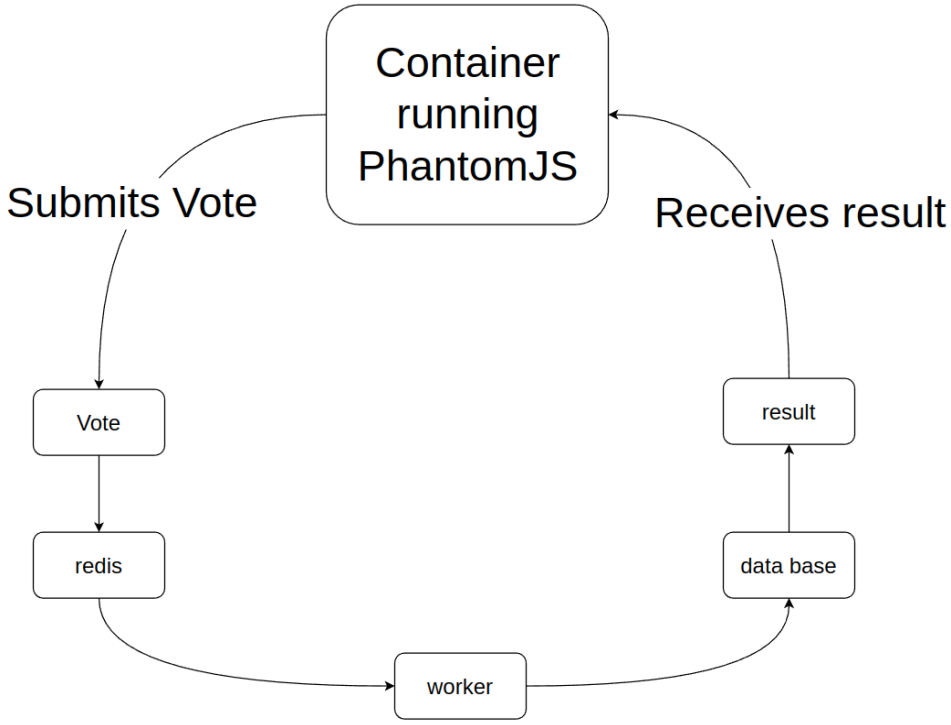
```
}
```

You should see a similar output in the Blue Ocean interface as the screenshot below. This indicates a successful execution of integration tests for the **`vote`** app:



E2E Tests

Examine the script **`example-voting-app/e2e.sh`** in the **`example-voting-app`** repository as well as the **`example-voting-app/e2e/docker-compose.yml`**. You do not need to understand all of the code, but reading the script and the Docker Compose setup it uses will give you a better understanding of how e2e tests are set up.



The `.env` File

`example-voting-app/e2e/docker-compose.yml` uses environment variables to decide which images to use to spin up the containers for the e2e tests.

Inside the `example-voting-app/e2e/.env` file you will find the following:

```
VOTE_IMAGE=xxxx/vote:latest
WORKER_IMAGE=xxxx/worker:latest
RESULT_IMAGE=xxxx/result:latest
```

Replace `xxxx` with your own Docker Hub user ID.

Setting Up E2E

Once the `.env` file is edited, run the following command from inside the `example-voting-app/e2e` which contains the `docker-compose.yml` file:

```
docker compose build
```

This will build the images necessary for the e2e test. It also ensures that any changes made to testing files are picked up by Docker Compose. After everything is built, run:

```
docker compose up -d
```

```
[+] Running 8/8
 ⠿ Network e2e_back-tier   Created                                0.1s
 ⠿ Network e2e_front-tier  Created                                0.1s
 ⠿ Container e2e-db-1      Started                                1.8s
 ⠿ Container e2e-redis-1   Started                                1.9s
 ⠿ Container e2e-worker-1  Started                                2.8s
 ⠿ Container e2e-vote-1    Started                                3.1s
 ⠿ Container e2e-result-1  Started                                3.4s
 ⠿ Container e2e-e2e-1     Started                                3.9s
```

To list your running containers to find their associated ports, run:

```
sudo docker ps
```

```
IMAGE                      PORTS
xxxxxxx/result:latest      0.0.0.0:49156->80/tcp, :::49156->80/tcp
xxxxxxx/vote:latest        0.0.0.0:49155->80/tcp, :::49155->80/tcp
```

Find the containers associated with the **`result:latest`** image and the **`vote:latest`** image. Visit **`IPADDRESS:VOTECONTAINERPORT`** and **`IPADDRESS:RESULTCONTAINERPORT`**. Submit a vote. This is manual e2e testing.

To automate the process, we will now spin up the e2e service, which will automatically submit a vote for you. The e2e service is listed in the `example-voting-app/e2e/docker-compose.yml`. Run:

```
docker compose run --rm e2e
```

```
-----------------
Current Votes Count: 13
-----------------

I: Submitting one more vote...

-----------------
New Votes Count: 14
-----------------
```

```
I: Checking if votes tally......

------------
Tests passed
------------
```

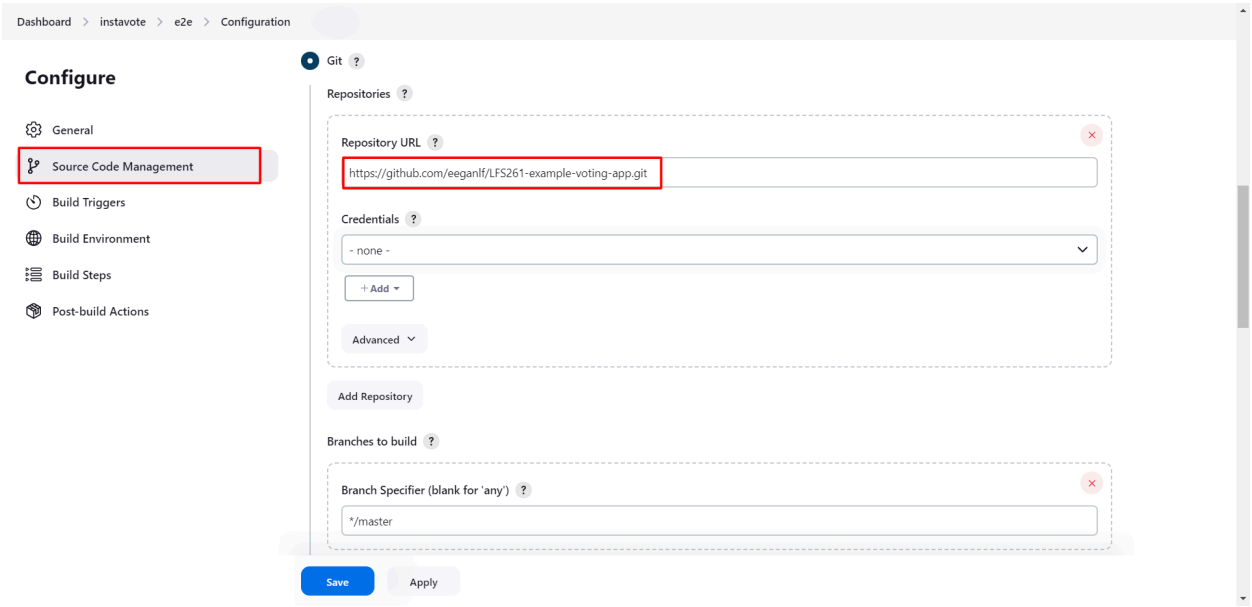
You should see something similar to the output above, though your counts will differ. If it fails, try running it again, and it should pass. This means that the application passed e2e testing. Shut the application down with the following command from inside `example-voting-app/e2e`:

```
docker compose down
```

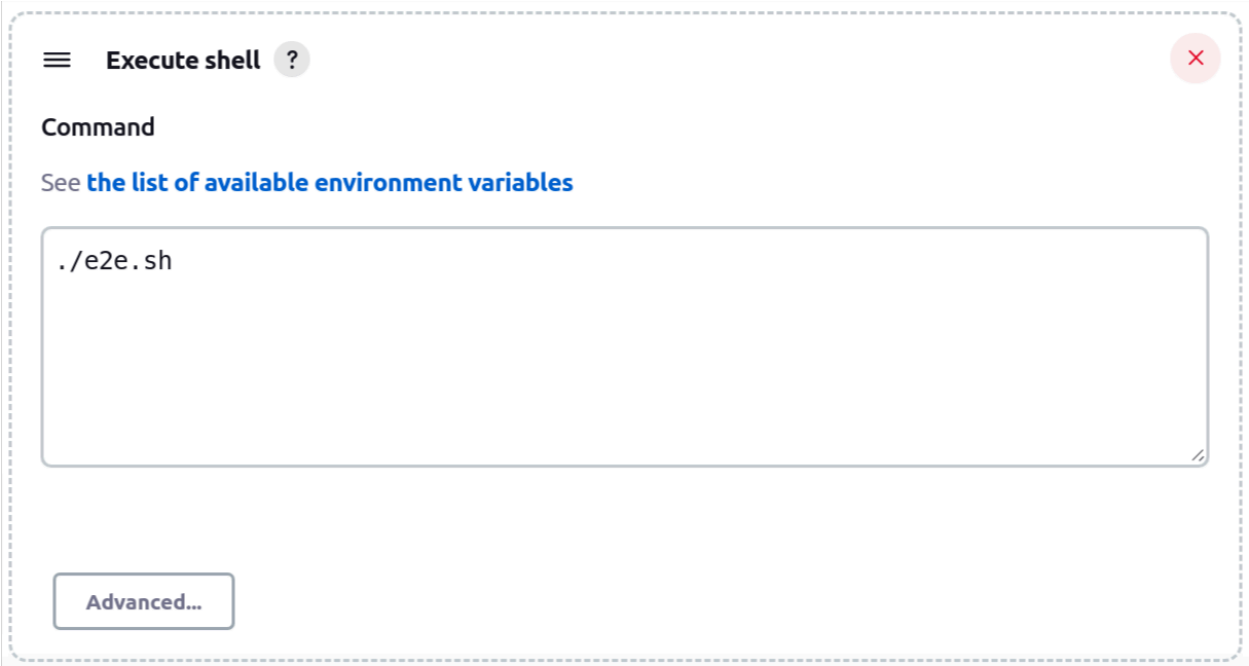
Using a container to test our application is the first building block to integrating the e2e test into our CI process, which we will do next.

Integrating E2E Tests with Jenkins

First, create a freestyle project called e2e in Jenkins. Add the Git repository:

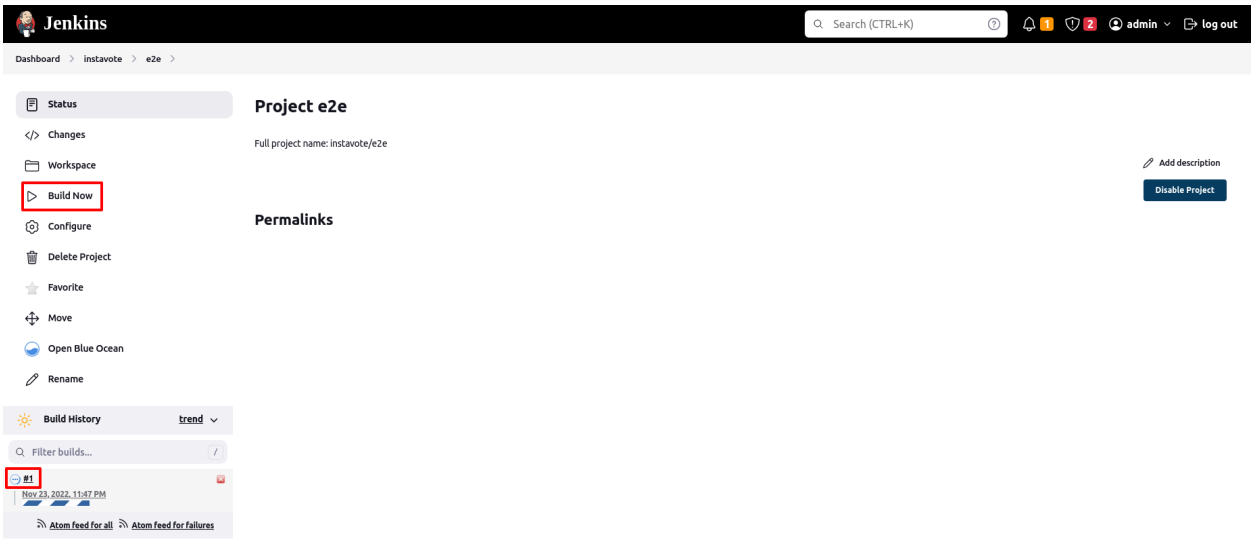


Inside the **Build Steps > Add build step > Execute shell > Command input** box paste: `./e2e.sh`

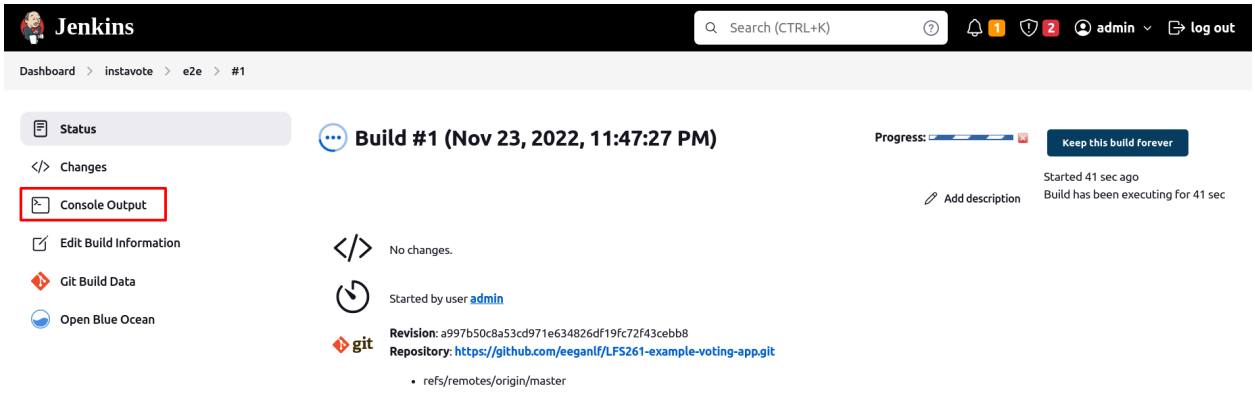


Save the configuration.

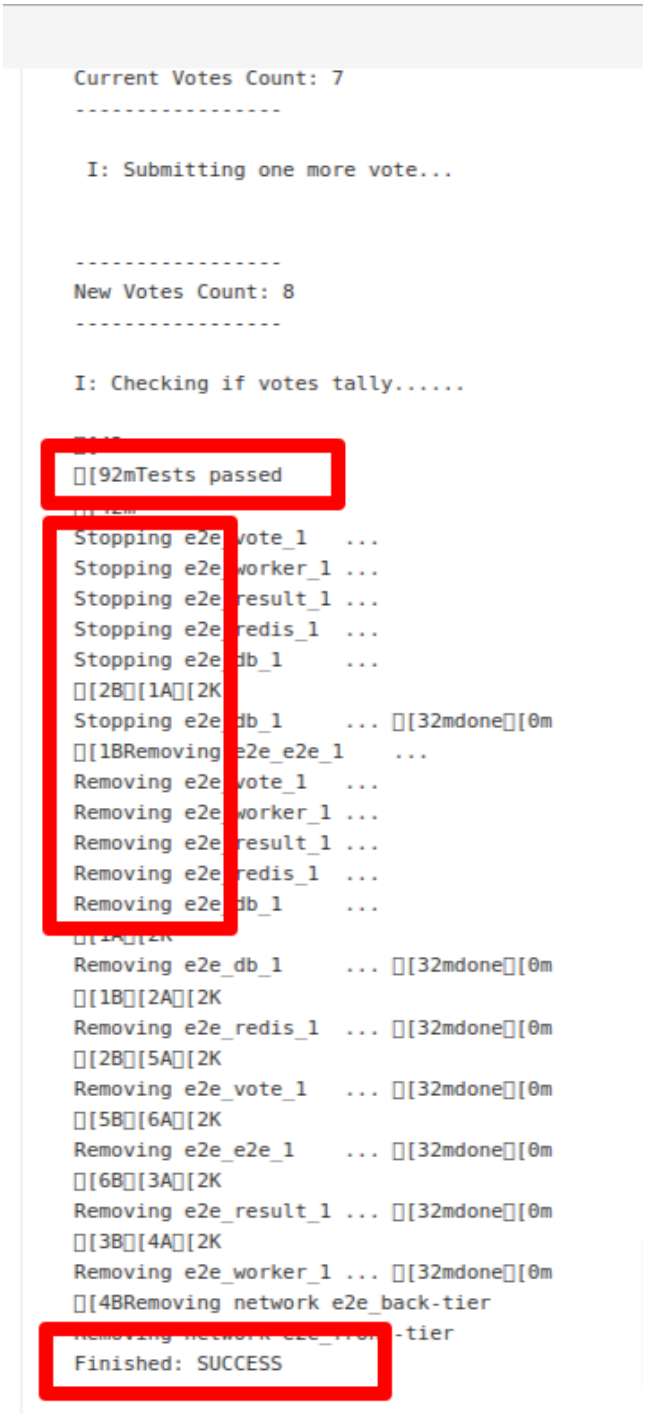
Run the job and click the build:



Click **Console output** to see the console output:



If the tests fail, try running the job twice more. You should ultimately see something like the following:



You should see that the tests passed and then you should see the containers being cleaned up with a `SUCCESS` message at the bottom of the output. You have successfully run e2e tests with Jenkins.

Summary

In this lab, we integrated Jacoco into Jenkins to obtain automatic testing coverage analysis. We then integrated SonarQube into Jenkins for automatic code analysis. Finally, we added integration and e2e tests into Jenkins.